1차 프로젝트 - 박소영

1. 프로젝트 주제
    1.1. 주 주제 : Nginx기반의 로드밸런싱 아키텍처 구축
    1.2. 세부 주제 : Nginx를 활용한 3-Tier 고가용성 로드밸런싱 시스템 설계 및 구현
    1.3. 주제 선정 배경
        벤치마킹 사이트
        사용자의 요구에 따른 기술적 반영 구획
2. 요구사항 정의
    2.1. 사용자 요구사항 (UR)
    2.2. 시스템 요구사항 (SR)
    2.3. 기능 요구사항 (FR)
    2.4. 비기능 요구사항
3. 프로젝트 목표
    3.1. 목표 개요
    3.2. 고가용성 구현을 위한 세부 목표
4. 시스템 아키텍처 설계서
    4.1. 아키텍처 개요(전체 구성)
    4.2. 세부 서비스별 설계
    4.3. 네트워크 설계
    4.4. 데이터 흐름 설계
5. 기타: 프로젝트 산출물
    5.1. 구현 서비스별 구현 파일 목록(목록번호 부여)
    5.2. 서비스별 구현 파일 첨부(목록번호 순서)

1. 프로젝트 주제

1.1. 주 주제 : Nginx기반의 로드밸런싱 아키텍처 구축

🛫**주제: "여행자 편의를 위한 물품 보관함 실시간 예약 플랫폼"**

1.2. 세부 주제 : Nginx를 활용한 3-Tier 고가용성 로드밸런싱 시스템 설계 및 구현

전철의 '사물함 예약'이라는 서비스를 목적에 두고, 이를 실현하기 위해 Nginx기반의 로드밸런싱 아키텍처를 3티어로 구현합니다.

1.3. 주제 선정 배경

💡 **▼ 벤치마킹 사이트**

또타라커 - 물품보관함

• **[또타라커 (Tota Locker)]**: 서울 지하철 물품보관함 앱. 서울교통공사의 공식 캐릭터인 '또타(TTOTA)'와 물품 보관함을 의미하는 '라커(Locker)'가 합쳐진 이름.

- **중앙 집중식 DB:** 서울 전역의 사물함 상태를 하나의 DB에서 관리합니다.
- **실시간 연동:** 사물함의 센서(물건이 들어있는지 확인)와 서버가 통신하여 실시간으로 앱에 상태를 전달합니다.
- **비대면/비회원 기반의 높은 접근성**: 앱을 통해 비밀번호나 QR코드로 비대면 이용이 가능합니다.

- **사용자 경험 기반의 문제 의식:** 여행지나 낯선 지역에서 대형 캐리어 등 무거운 짐을 소지한 채 이동해야 하는 물리적 불편함을 해소하고자 합니다.

- **물품 보관함의 필요성:** 역내 사물함을 이용해 본 개인적인 경험을 바탕으로, 사용자 편의를 극대화한 디지털 예약 플랫폼의 필요성을 느꼈습니다.
  - **간편한 예약:** 복잡한 가입 절차 없이 비회원 전형으로 연락처와 비밀번호만으로 즉시 사물함을 선점할 수 있게 합니다.
  - **실시간 현황 조회:** 역에 도착하기 전, 미리 스마트폰으로 빈 사물함이 있는지 확인하여 헛걸음 하지 않도록 돕습니다.

▼ 사용자의 요구에 따른 기술적 반영 구획

사용자의 요구	기술적 반영 (Implementation)
비회원 기반 접근성	**FastAPI Reserve 서비스:** 연락처와 비밀번호만으로 즉시 예약 가능한 로직 구현
실시간 조회	**Nginx 기반 로드밸런싱:** Ingress 도메인과 연동된 Nginx가 외부 요청의 단일 진입점이 되어, URL 경로에 따라 최적의 서비스 파드로 트래픽을 분산함으로써 대규모 동시 접속 상황에서도 중단 없는 조회 환경을 제공
예약 데이터 보호	**NFS 중앙 집중식 스토리지:** 모든 서비스 로그와 DB 백업 데이터를 독립된 NFS 서버에 실시간 기록하여, 멀티 노드 클러스터 환경에서도 데이터의 영속성과 안전한 관리를 실현
네트워크 가용성	**MetalLB / Ingress:** MetalLB를 통해 외부 접속용 External IP를 확보하고, Ingress Controller가 도메인 주소를 기반으로 요청을 수신하여 내부 ClusterIP 서비스로 트래픽을 정교하게 배분

---

2. 요구사항 정의

▼ 2.1. 사용자 요구사항 (UR)

1. 사물함 현황 조회 (View)

UR-1.1 (역 선택): 사용자는 메인 화면에서 원하는 역(서울역, 부산역, 대구역)을 선택할 수 있다.

UR-1.2 (사물함 배치도): 사용자는 선택한 역의 사물함 배치도(3x4 형태의 12개 사물함)를 시각적으로 확인할 수 있다.

UR-1.3 (사물함 색상): 사용자의 빠른 파악을 위한 사물함의 상태는 색상(초록: 사용 가능, 빨간: 사용 중)을 통해 직관적으로 구분된다.

2. 사물함 예약 및 이용 (Reserve)

UR-2.1 (예약 페이지 이동): 비회원인 사용자는 '사용 가능' 상태인 사물함을 클릭하여 예약 페이지로 이동할 수 있어야 한다.

UR-2.2 (예약 정보 기입): 예약 시, 사용자는 본인 확인을 위한 연락처와 사물함 개폐를 위한 비밀번호(4자리)를 설정해야 한다.

UR-2.3 (예약 확정): 예약이 완료되면 화면에 표시된 확정 메세지를 통해 사용자는 예약이 확정됐음을 확인할 수 있다.

UR-2.4 (예약 조회): 사용자는 본인이 예약한 정보를 사물함 번호와 비밀번호를 입력하여 예약 정보를 조회할 수 있어야 한다.

▼ 2.2. 시스템 요구사항 (SR)

1. 인프라 및 네트워크 (K8s & Nginx)

SR-1.1 (MSA 구성): 사물함 조회, 예약, 관리 기능을 독립된 FastAPI 서비스 파드(`locker-view`, `locker-reserve`, `locker-admin`)로 구성하고 K8s Deployment로 배포한다.

SR-1.2 (L2 로드밸런싱): MetalLB를 활용하여 클러스터 외부 접속을 위한 단일 접점(External IP: 172.16.0.110)을 확보한다.

SR-1.3 (도메인 기반 라우팅): Ingress Controller가 호스트 이름(`sy-locker-view.com`)에 따라 각 마이크로서비스로 트래픽을 정교하게 라우팅하도록 설정한다.

SR-1.4 (입구 고가용성): Bastion 노드에 HAProxy와 Keepalived를 설정하여 외부 요청 수신부의 이중화 및 장애 조치 체계를 구축한다.

2. 데이터 가용성 및 저장소 (DB & NFS)

SR-2.1 (DB 복제 및 통신): MySQL Master-Slave 복제 구조를 구축하고, K8s Endpoints를 통해 클러스터 외부 DB 노드와 안전하게 통신한다.

SR-2.2 (데이터 영속성): NFS 서버를 `nfs-pv`, `nfs-log-pv`와 연동하여 파드의 재시작이나 노드 장애와 관계없이 서비스 데이터와 로그가 영구 보존되는 환경을 구축한다.

SR-2.3 (컨테이너 배포): FastAPI 애플리케이션은 Dockerfile 빌드를 통해 이미지화하며, 레지스트리를 거쳐 각 워커 노드에 일관되게 배포한다.

▼ 2.3. 기능 요구사항 (FR)

1. 현황 조회 및 웹 서비스 (Locker-View)

FR-1.1 (정적 콘텐츠 제공): Nginx 파드는 사용자가 `/` 경로로 접속 시 NFS에 저장된 최신 HTML 파일을 웹 브라우저에 출력한다.

FR-1.2 (사물함 상태 조회): 특정 역의 사물함 상태 정보를 DB에서 조회하여 JSON 형식으로 실시간 반환한다.

2. 예약 처리 기능 (Locker-Reserve)

FR-2.1 (사물함 예약): 사용자가 번호와 비밀번호 입력 시 상태를 `occupied`로 업데이트하고 예약 정보를 DB에 저장한다.

FR-2.2 (중복 예약 방지): 이미 사용 중인 사물함에 대해 예약 시도가 있을 경우 에러 메시지를 반환하여 데이터 충돌을 방지한다.

3. 관리자 기능 (Locker-Admin)

FR-3.1 (사물함 강제 제어): 관리자는 특정 사물함의 이용을 강제로 종료하고 상태를 `available`로 초기화할 수 있다.

FR-3.2 (인가 제어): 인가되지 않은 일반 사용자가 관리 URL이나 기능에 접근할 경우 `403 Forbidden`을 반환하고 접근을 차단한다.

▼ 2.4. 비기능 요구사항

1. 가용성 및 신뢰성 (Availability & Reliability)

NFR-1.1 (서비스 고가용성): `Keepalived`와 `HAProxy` 이중화 구성을 통해 로드밸런서 장애 시에도 서비스 중단 없이 VIP(Virtual IP)를 승계하여 고가용성을 유지한다.

NFR-1.2 (데이터 무결성): MySQL `Master-Slave Replication`을 통해 실시간 데이터 복제본을 유지하며, 주 노드 장애 시 데이터 손실을 최소화한다.

2. 보안성 (Security)

NFR-2.1 (접근 통제): 모든 관리 접속은 Bastion Host를 거쳐야 하며, 허용된 IP 대역 외의 관리자 경로 접근은 Ingress 수준에서 원천 차단한다.

NFR-2.2 (네트워크 격리): 서비스 간 통신은 클러스터 내부망(`ClusterIP`)을 사용하여 외부 노출을 최소화한다.

3. 모니터링 및 장애 복구 (Observability)

NFR-3.1 (통합 관제 및 로깅): Prometheus/Grafana로 리소스 상태를 시각화하고, Loki를 통해 모든 예약/관리 이력을 중앙 집중식으로 기록하고 추적한다.

NFR-3.2 (지속적 배포): Docker 이미지 기반의 배포 방식을 채택하여 Git을 통해 개발 환경과 운영 환경의 일관성을 유지하고 배포 자동화의 기반을 마련한다.

NFR-3.3 (정기 백업 및 복구): Crontab 스케줄러를 통해 매일 정해진 시간에 DB 전체 덤프(.sql)를 생성한다. 생성된 백업 파일은 Hostpath를 통해 클러스터 NFS 외부 스토리지에 격리 저장하며, 시스템 전체 장애 시 이를 통해 즉시 복구한다.

---

3. 프로젝트 목표

▼ 3.1. 목표 개요

3.1.1. 고가용성 시스템 구축

- **서비스 독립성 확보:** 사물함 조회(View), 예약(Reserve), 관리(Admin) 기능을 독립된 FastAPI 서비스(MSA)로 분리하여 K8s Deployment로 배포함으로써 특정 서비스 장애가 전체로 확산되는 것을 방지합니다.
- **컴포턴트 다중화:** 사용자 요청을 처리하는 FastAPI 애플리케이션은 K8s Deployment를 통해 다중화(Replicas)하여 트래픽 부하를 분산하고, 파드 장애 시 K8s의 자가 치유(Self-healing) 기능을 통해 무중단 서비스를 지향합니다.
- **데이터 연속성 보장:** 데이터베이스는 MySQL Master-Slave 복제를 통해 장애 발생 시에도 데이터 유실 없이 서비스를 지속할 수 있는 인프라를 지향합니다. Slave DB는 실시간 서비스 부하를 받지 않는 독립된 백업 노드로 운영하여 데이터의 영속성을 극대화합니다.

3.1.2. 부하 분산을 통한 성능 향상

- **지능적 트래픽 관리:** MetalLB L2 로드밸런서를 통해 외부 접점(`172.16.0.110`)을 단일화하고, Nginx Ingress Controller를 통해 도메인 및 URL 경로 기반의 L7 라우팅을 수행하여 시스템 전반의 응답 속도를 최적화합니다.
- **보안 계층 분리:** 모든 관리 트래픽은 Bastion 호스트를 경유하도록 설계하고, `/admin` 경로는 IP 필터링을 적용하여 성능 향상과 동시에 강력한 보안성을 확보합니다.

3.1.3. 장애 대응 및 복구 체계

- **자원 관리 일원화:** 분산된 K8s 워커 노드 환경에서도 모든 서비스 로그와 DB 백업 데이터를 NFS 중앙 저장소에 실시간 보관하여 실무적인 데이터 보존 체계를 구현합니다.
- **실시간 데이터 무결성:** MySQL Binary Log 기반의 실시간 동기화를 통해 데이터 일관성을 유지하며, `Crontab` 스케줄러를 활용해 주기적으로 DB 덤프 파일을 생성하여 시스템 전체 위기 상황에 대비합니다.

▼ 3.2. 고가용성 구현을 위한 세부 목표

3.2.1. 로드밸런서 설정

- **MetalLB:** 외부 접속용 VIP(`172.16.0.110`)를 할당하여 클러스터 진입점을 고정합니다.
- **Ingress:** 도메인(`sy-locker-view.com`) 기반으로 서비스별 경로(`/`, `/reserve`, `/admin` 등)를 분기하여 각 FastAPI ClusterIP 서비스로 연결합니다.

3.2.2. 클러스터링 구성

- **구조:** 1개의 Master 노드와 2개의 Worker 노드로 구성된 K8s 클러스터 환경을 구축합니다.
- **네트워크:** Calico CNI를 적용하여 파드 간 통신을 최적화하고, `Endpoints`를 통해 서비스와 파드 간의 가용 IP를 실시간 매핑합니다.

3.2.3. 데이터 복제(Replication) 구성

MySQL Master-Slave 복제를 통해 장애 발생 시에도 데이터 유실 없이 서비스를 지속할 수 있는 인프라를 지향합니다. Slave DB는 실시간 서비스 부하를 받지 않는 독립된 백업 노드로 운영하여 데이터의 영속성을 극대화합니다.

3.2.4. 모니터링 및 로깅 시스템 구축

- **메트릭 시각화:** `Prometheus`로 수집된 노드/파드 자원 지표를 `Grafana` 대시보드에서 실시간 모니터링합니다.
- **중앙 로깅:** `Loki`를 통해 모든 마이크로서비스의 로그를 통합 관리하며, 이 로그 데이터의 원본은 default.conf 파일을 통해 NFS에 영구 기록하여 사후 장애 분석 근거를 제공합니다.

---

4. 시스템 아키텍처 설계서

▼ 4.1. 아키텍처 개요(전체 구성)

4.1.1. 3-Tier 아키텍처 설명

▼ VM의 설정 (8개)

네트워크 구분	할당 대역 (CIDR)	용도
Bridge (User) IP 대역	`10.0.105.0/24`	외부 접속 및 Bastion 관리용
Internal (Server) IP 대역	`172.16.0.0/24`	서버 간 내부 통신용 (Host-only)

VM (가상머신)	구분	Bridged IP (외부/관리)	Host-only IP (내부망)	비고
Bastion Host	Bastion	10.0.105.1/16	172.16.0.2/24	외부 접속 GW
Bastion Host-slave	Bastion	10.0.105.2/16	172.16.0.11/24	이중화(HA) 구성
NFS	Storage	-	172.16.0.3	공유 스토리지
k8s-m (Master)	K8s	-	172.16.0.5	쿠버네티스 제어
node1 (Worker1)	K8s	-	172.16.0.6	워커 노드 1
node2 (Worker2)	K8s	-	172.16.0.7	워커 노드 2
db-m (Master)	Database	-	172.16.0.8	DB 메인 서버
db-s (Slave)	Database	-	172.16.0.9	DB 복제 서버

▼ JumpProxy 설정 (C:\Users\SY\.ssh\cofig)

```
# 공통 설정 (변수처럼 상단에 선언 가능)
Host *
  AddKeysToAgent yes
  IdentityFile ~/.ssh/id_rsa
  User sy


# --- 1. 대문 (Bastion Hosts) ---
```

```
# 메인 배스천
Host bastion
  HostName 10.0.105.1
  Port 22

# 백업 배스천
Host bastion2
  HostName 172.16.0.11
  Port 22


# --- 2. 내부 서버들 (메인 배스천 경유) ---
# bastion이 안 되면 bastion2를 시도합니다(순차적 시도)
Host nfs
  HostName 172.16.0.3
  ProxyJump bastion,bastion2

Host k8s-m
  HostName 172.16.0.5
  ProxyJump bastion,bastion2

Host node1
  HostName 172.16.0.6
  ProxyJump bastion,bastion2

Host node2
  HostName 172.16.0.7
  ProxyJump bastion,bastion2


Host db-m
  HostName 172.16.0.8
  ProxyJump bastion,bastion2

Host db-s
  HostName 172.16.0.9
  ProxyJump bastion,bastion2
```

▼ 트리 구조

- **k8s-m**: 컨테이너화된 애플리케이션의 배포 및 운영 설정

```
sy@k8s-m: ~/project/k8s-manifests
sy@k8s-m:~/project$ tree
.
├── db-configs
│   └── db_backup.sh
├── k8s-manifests
│   ├── db-backup-hostpath.yaml
│   ├── deployment.yaml
│   ├── fastapi-services.yaml
│   ├── grafana-ingress.yaml
│   ├── ingress.yaml
│   ├── mariadb-external.yaml
│   ├── metallb.yaml
│   ├── nfs-log-pv-pvc.yaml
│   ├── nginx-conf.yaml
│   └── pv-pvc.yaml
├── nginx
│   ├── 403.html
│   └── index.html
├── service-admin
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── service-locker
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
└── service-reserve
    ├── Dockerfile
    ├── main.py
    └── requirements.txt

7 directories, 22 files
```

관련 디렉토리	담당 역할 (Role)	주요 역할 및 상세 내용
`k8s-manifests/`	**1. Ingress (접속 및 라우팅)**	외부 도메인(`sy-locker-view.com`) 접속 처리, SSL 인증 및 경로 분기
	2. Service (네트워크 및 로드밸런싱)	클러스터 내부 IP 할당 및 MetalLB 기반 외부 고정 IP(VIP) 제공
	3. Storage (저장 연결)	외부 NFS 서버와 Pod를 연결하는 영구 볼륨 (`PV/PVC`) 통로 구축
`service-*/`	**4. Application (서비스 구성)**	`admin` , `locker` , `reserve` 각 서비스별 FastAPI 구동 및 DB 연동 수행
`nginx/`	**5. Proxy (웹 서버)**	`index.html` , `403.html` 등 사용자에게 직접 보여지는 정적 웹 자산 서빙 및 백엔드 API 서버로의 요청 전달
`db-configs/`	**6. Backup (데이터 보호)**	DB 스냅샷 추출 스크립트 (`db_backup.sh`) 관리 및 외부 NFS 전송 자동화

- **nfs**: 클러스터와 분리된 별도의 데이터 저장소

```
sy@nfs: /mnt
sy@nfs:/mnt$ tree
.
├── DATA1
│   └── nginx-logs
│       ├── access.log
│       ├── error.log
│       ├── sy_access.log
│       └── sy_error.log
└── DATA2
    ├── k8s-web
    │   ├── 403.html
    │   └── index.html
    ├── master
    │   └── data
    │       └── master_backup.sql
    └── slave
        └── data
            ├── db_full_20260318_051625.sql
            ├── db_full_20260318_063513.sql
            ├── db_full_20260319_030001.sql
            ├── db_full_20260320_030001.sql
            └── db_full_20260320_032824.sql

9 directories, 12 files
```

구분	파티션 (장치)	마운트 경로	주요 역할 (Role)
DATA1	`/dev/sdb1`	`/mnt/DATA1`	**Log 전용** (Nginx, FastAPI)
DATA2	`/dev/sdb2`	`/mnt/DATA2`	**데이터/백업 전용** (Web, DB Dump)

▼ 쿠버네티스 설정
▼ 쿠버네티스 구성 캡쳐

```
sy@k8s-m:~/project$ kubectl get pods -n locker-view-ns
NAME                                    READY   STATUS    RESTARTS      AGE
backup-monitor-65c65f5d97-kjdt5         1/1     Running   1 (3d1h ago)  3d20h
locker-admin-deploy-98887cb48-5vvsj     1/1     Running   1 (3d1h ago)  3d17h
locker-reserve-deploy-7c5c5f9b4-7md9c   1/1     Running   1 (3d1h ago)  3d16h
locker-view-deploy-5f78ff6557-2jsvx     1/1     Running   1 (3d1h ago)  3d17h
locker-view-deploy-5f78ff6557-hnmp8     1/1     Running   1 (3d1h ago)  3d16h
nginx-deploy-lab-66d97945dd-9dmhp       1/1     Running   0             2d18h
nginx-deploy-lab-66d97945dd-mdjnh       1/1     Running   0             2d18h
nginx-deploy-lab-66d97945dd-pndc9       1/1     Running   0             2d18h
```

파드

```
sy@k8s-m:~/project$ kubectl get deployment -n locker-view-ns
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
backup-monitor          1/1     1            1           4d19h
locker-admin-deploy     1/1     1            1           3d22h
locker-reserve-deploy   1/1     1            1           3d22h
locker-view-deploy      2/2     2            2           3d22h
nginx-deploy-lab        3/3     3            3           5d20h
```

디플로이먼트

```
sy@k8s-m:~/project$ kubectl get svc -n locker-view-ns
NAME                  TYPE           CLUSTER-IP      EXTERNAL-IP                      PORT(S)    AGE
admin-service-svc     ClusterIP      10.103.216.102  <none>                           8003/TCP   3d22h
grafana-internal-svc  ExternalName   <none>          grafana.default.svc.cluster.local <none>    3d
locker-service-svc    ClusterIP      10.107.164.151  <none>                           8001/TCP   3d22h
mariadb-master-svc    ClusterIP      10.101.208.146  <none>                           3306/TCP   3d19h
nginx-clusterip-svc   ClusterIP      10.98.9.171     <none>                           80/TCP     5d20h
reserve-service-svc   ClusterIP      10.98.49.219    <none>                           8002/TCP   3d22h
```

서비스

```
sy@k8s-m:~/project$ kubectl get endpoints -n locker-view-ns
NAME                  ENDPOINTS                                                     AGE
admin-service-svc     10.244.166.187:8003                                           3d22h
locker-service-svc    10.244.166.137:8001,10.244.166.155:8001                       3d22h
mariadb-master-svc    172.16.0.8:3306                                               3d17h
nginx-clusterip-svc   10.244.104.14:80,10.244.166.175:80,10.244.166.176:80          5d20h
reserve-service-svc   10.244.166.188:8002                                           3d22h
```

엔드포인트

```
sy@k8s-m:~/project$ kubectl get pv,pvc -n locker-view-ns
NAME                                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/nfs-log-pv   5Gi        RWX            Retain           Bound    locker-view-ns/nfs-log-pvc
               <unset>                          5d20h
persistentvolume/nfs-pv       10Gi       RWX            Retain           Bound    locker-view-ns/nfs-pvc
               <unset>                          5d20h

NAME                                STATUS   VOLUME      CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIB
UTESCLASS   AGE
persistentvolumeclaim/nfs-log-pvc   Bound    nfs-log-pv  5Gi        RWX                           <unset>
            5d20h
persistentvolumeclaim/nfs-pvc       Bound    nfs-pv      10Gi       RWX                           <unset>
            5d20h
```

PV & PVC

```
sy@k8s-m:~/project$ kubectl get ingress -n locker-view-ns
NAME                 CLASS   HOSTS                                  ADDRESS        PORTS     AGE
grafana-ingress      nginx   grafana.sy-locker-view.com             172.16.0.110   80, 443   3d
lab-ingress          nginx   sy-locker-view.com,www.sy-locker-view.com  172.16.0.110   80, 443   5d20h
```

인그레스

이 쿠버네티스는 k8s 매니페스트 파일(.yaml)을 배포해 구성합니다.

항목	주요 역할	구성 상세	핵심 특징
POD	실제 서비스 컨테이너	Nginx, Locker-View/Reserve/Admin, Backup-Monitor	`10.244.x.x` (Calico 기반 Pod 네트워크)
SVC	내부 서비스 노출	ClusterIP	클러스터 내부 통신 전용 보안 강화
PV / PVC	영구 데이터 저장	**nfs-log-pv:** 로그 데이터 **nfs-pv:** 웹 정적 자산 및 DB 백업	**NFS Server:** `172.16.0.3` 상태: **Bound**
MetalLB	외부 고정 IP(VIP) 할당	클러스터 외부 진입점용 VIP 제공	**External IP:** `172.16.0.110` (프로젝트 전용 VIP)
Endpoints	서비스-파드 실시간 연결	FastAPI, DB, Cluster IP의 타겟 IP 리스트	트래픽 부하 분산 (Load Balancing) 대상 관리
Ingress	외부 도메인 라우팅 (L7)	**Hosts:** sy-locker-view.com, grafana.sy-locker-view.com	**172.16.0.110** (80, 443 포트) 도메인 분기
deployment	파드 관리 및 배포 전략	Replicas 유지, 롤링 업데이트	자가 치유(Self-Healing) 및 가용성 유지

▼ 물리적 구성도 (Physical Architecture)

```
[ External User / Admin ]
          |
          ▼ (Bridge 망: 10.0.105.x)
┌──────────────────────────────────────────────────────────┐
```

```
│  Gateway Zone (Bastion HA)                          │
│  - bastion-m (10.0.105.1), bastion-s (10.0.105.2)   │
└──────────────────────────────────────────────────────┘
          │
          │ (Internal 서비스망: 172.16.0.x)
          │
    ┌─────┴──────────────┬──────────────────────┬──────────────┐
    │                    │                      │              │
    ▼                    ▼                      ▼              ▼

┌──────────────┐   ┌──────────────────┐   ┌──────────────┐   ┌──────────────┐
│ K8s Cluster  │   │ Database Group (VM) │   │ Storage (VM) │   │ 모니터링/기타 │
│              │   │                  │   │              │   │              │
│ - k8s-m      │   │ - db-m (172.16.0.8) │   │ - NFS Server │   │ - Grafana    │
│ - node1      │   │ - db-s (172.16.0.9) │   │ (172.16.0.3) │   │ (External)   │
│ - node2      │   └──────────────────┘   └──────────────┘   └──────────────┘
└──────────────┘            (Replication)            (Logs/Data)
```

▼ 논리적 구성도 (Logical Architecture)

```
[ User Request: sy-locker-view.com ]
          │
          ▼ [L4] MetalLB (VIP: 172.16.0.110)
┌──────────────────────────────────────────────────────────┐
│ [L7 Entrance] Ingress Controller (Nginx Ingress)          │
│ - SSL: sy-locker-tls-secret                               │
│ - Path Routing: / (Web: Nginx), /api/* (WAS: FastAPI)     │
└──────────────────────────────────────────────────────────┘
           │                   │                    │
           │ (/)               │ (/api/reserve)     │ (/api/admin)
           ▼                   ▼                    ▼

┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ [SVC] nginx-svc │   │ [SVC] reserve-svc│   │ [SVC] admin-svc │
└──────────────┘   └──────────────┘   └──────────────┘
           │
           │ (Load Balancing)    │                  │
   ┌───────┴───────┐             │                  │
   ▼       ▼       ▼             ▼                  ▼

┌───┐    ┌───┐    ┌───┐         ┌───┐              ┌───┐
│Pod│    │Pod│    │Pod│         │Pod│              │Pod│
└───┘    └───┘    └───┘         └───┘              └───┘
 [ Nginx Replicas: 3 ]     [ Reserve: 1 ]     [ Admin: 1 ]
(Static Web / Proxy)       (Reservation)      (Management)
           │                   │                  │
           └─────────────────────┴──────────────────┘
                     │                    │
                     │                    │
     ┌───────────────┴────────────────────┴──────────────┐
     │           [Resource Connectivity Layer]            │
     ├────────────────────────────────────────────────────┤
     │  1. Storage (NFS Protocol - 172.16.0.3)            │
     │     - Static Assets: nfs-pvc (DATA2/k8s-web)       │
     │     - System Logs: nfs-log-pvc (DATA1/nginx-logs)  │
     │----------------------------------------------------│
     │  2. Database (TCP: 3306 - 172.16.0.8)              │
     │     - Service Name: mariadb-master-svc (Endpoints) │
```

```
│      - Master: Write / Slave: Backup 전용              │
└──────────────────────────────────────────────────────┘
```

MetalLB → Ingress → MSA(FastAPI) → Master-Slave DB → NFS

1. **네트워크 유입 계층 (L4 → L7)**
   - **MetalLB(L4 Load Balancer):** 가상 IP(172.16.0.110)를 클러스터 외부에 노출시켜 외부 트래픽이 들어올 수 있는 실제 통로를 열어줍니다.
   - **Ingress(L7 Controller):** 들어온 트래픽의 '주소(URL)'를 확인합니다. `sy-locker-view.com/` 이면 웹 서버로, `/api/*` 면 백엔드 서버로 사용자를 안내합니다.
     - **시크릿(Secret):** 보안을 위해 네임스페이스에 할당하며, 인그레스 지점에서 TLS 암호화 통신을 전담함으로써 내부 서비스들이 복호화 연산 부하 없이 로직 처리에만 집중할 수 있도록 돕습니다.

```
sy@k8s-m:~$ kubectl get secret -n locker-view-ns
NAME                  TYPE                  DATA   AGE
sy-locker-tls-secret  kubernetes.io/tls     2      6d5h
```

2. **애플리케이션 계층 (Web-WAS 분리)**
   - **Nginx (Web/Proxy):** NFS(`DATA2/k8s-web`)에서 정적 자산(index.html 파일)을 읽어 사용자에게 직접 서빙합니다.
     - **리버스 프록시:** API 요청이 들어오면 내부의 FastAPI 서비스로 전달합니다.
   - **FastAPI (MSA):** `admin`, `locker`, `reserve` 등 기능별로 서비스를 분리하여 특정 기능에만 부하가 몰릴 경우를 대비해 독립적으로 확장했습니다.

3. **데이터 및 저장소 계층 (DB & Storage)**
   - **Master-Slave DB (MariaDB/MySQL):** `db-m` 은 쓰기/서비스용, `db-s` 는 복제(Replication)를 통해 백업이나 조회 전용으로 사용하여 부하를 분산하고 고가용성을 확보합니다.
   - **NFS 백업 (Network File System):** 쿠버네티스 내부의 데이터(로그, 웹 소스, DB 백업본)를 클러스터 외부의 독립적인 NFS 서버에 저장함으로써, Pod가 삭제되어도 데이터가 유지되는 영속성을 구현했습니다. 동일한 저장소(`DATA1`, `DATA2`)를 바라보게 하여 중앙 집중 관리를 통해 데이터 일관성을 유지합니다.

4.1.2. 로드밸런서 역할

> MetalLB는 외부 단일 접점인 엔드포인트에 VIP(172.16.0.110)를 만들고, Ingress는 이렇게 로드밸런서가 구축한 VIP 대역에 도메인 주소를 할당해 트래픽을 처리합니다.

- **MetalLB (L4: IP/Port)**
  - **VIP 할당:** 클러스터 외부와 통신 가능한 가상 IP(`172.16.0.110`)를 생성하고, 이를 외부 네트워크에 VIP를 할당하여 실제 서비스로 들어오는 입구를 마련합니다.
  - **트래픽 전달:** 특정 도메인이나 경로에 상관없이, 해당 IP의 80/443 포트로 들어오는 모든 트래픽을 인그레스 컨트롤러(Ingress Controller)로 전달합니다.
- **Ingress (L7: 도메인/경로)**

- **도메인 기반 분배:** MetalLB가 열어준 통로를 통해 들어온 요청의 'HTTP 헤더'를 분석합니다. (`sy-locker-view.com` / `grafana.sy-locker-view.com`)
- **세부 경로 필터링:** 동일 도메인 내에서도 `/` 요청은 정적 웹 서버(Nginx)로, `/api/*` 요청은 백엔드(FastAPI) 서비스를 분류하여 배송하는 역할을 수행합니다.

▼ 4.2. 세부 서비스별 설계

4.2.1. 프론트엔드 로드밸런서

MetalLB로부터 할당받은 VIP(172.16.0.110)를 통해 접속을 허용하며, Nginx는 `/` 경로 요청 시 NFS의 정적 자원 서빙(Web)과 리버스 프록시(WAS 연결) 역할을 동시에 수행합니다.

▼ k8s-m의 리버스 프록시 설정(default.conf)

```
# nginx-conf.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: locker-view-ns
data:
  default.conf: |
    # [핵심] 새로운 로그 포맷 정의 (맨 앞에 실제 IP가 오도록 설정)
    log_format sy_special '$remote_addr - $remote_user [$time_local] "$reques
t" '
                          '$status $body_bytes_sent "$http_referer" '
                          '"$http_user_agent" "$http_x_forwarded_for"';

    server {
        listen 80;
        server_name sy-locker-view.com;
        charset utf-8;

        # [진짜 IP 추출 설정]
        # 모든 사설 대역(Calico, Node, MetalLB)을 신뢰 대역으로 추가합니다.
        set_real_ip_from 10.0.0.0/8;
        set_real_ip_from 172.16.0.0/12;
        set_real_ip_from 192.168.0.0/16;
        real_ip_header X-Forwarded-For;
        real_ip_recursive on;

        # [수정] 위에서 정의한 sy_special 포맷을 사용합니다.
        access_log /var/log/nginx/sy_access.log sy_special;
        error_log  /var/log/nginx/sy_error.log;

        location / {
            root /usr/share/nginx/html;
            index index.html;
            try_files $uri $uri/ /index.html;
        }

        location /api/lockers/ {
            proxy_pass http://locker-service-svc:8001;
```

```
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        location /api/reserve {
            proxy_pass http://reserve-service-svc:8002;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        location /api/admin/ {
            proxy_pass http://admin-service-svc:8003/api/admin/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        error_page 403 /403.html;
        location = /403.html {
            root /usr/share/nginx/html;
            internal;
        }
    }
```

▼ **k8s-m의 keepalived.conf 설정**

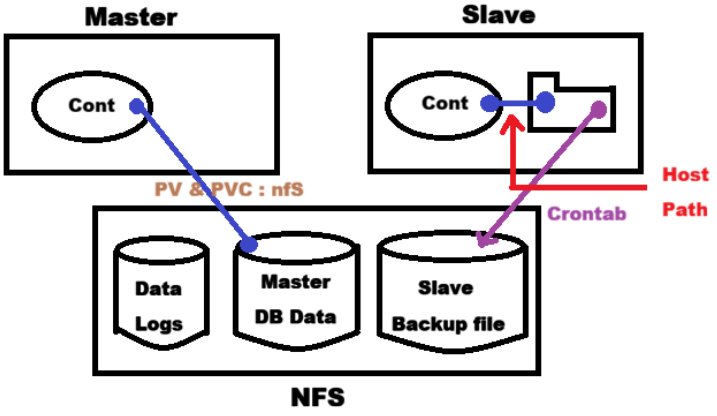
```
# /etc/keepalived/keepalived.conf

# [외부망용 - 서비스 접속용]
vrrp_instance VI_1 {
    state MASTER
    interface ens33        # 본인의 네트워크 인터페이스
    virtual_router_id 51   # 동일 네트워크 내 유일한 값
    priority 101
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1234
    }
    virtual_ipaddress {
        172.16.0.200       # 사용할 VIP(호스트 PC가 접속할 IP)
    }
}


# [내부망용 - 노드 게이트웨이용]
vrrp_instance VI_INTERNAL {
    state MASTER
    interface ens33          # 내부망 인터페이스 (동일할 경우 ens33 유지)
    virtual_router_id 52     # 외부용(51)과 겹치지 않게 설정
    priority 101
```

```
    advert_int 1
    virtual_ipaddress {
        172.16.0.100          # 내부 노드들이 바라볼 게이트웨이 IP
    }
}
```

1. **고가용성 확보 (Self-Healing):** `Deployment`의 Nginx 파드는 3개로, 만약 하나의 파드에 장애가 발생해도 나머지 2개가 즉시 서비스를 이어받습니다. 이 때 쿠버네티스가 자동으로 새 파드를 생성하여 가용성을 유지합니다.
2. **보안 강화 (Bastion & Proxy):** 모든 내부 서비스는 ClusterIP를 통해 노출되므로 클러스터 외부에서는 직접 접근이 불가능합니다. 오직 검증된 인그레스 도메인만을 통과하게 하여 내부 인프라 구조를 안전하게 보호합니다.
3. **데이터 일관성:** 여러 개의 Nginx 파드가 생성되어도 모두 동일한 NFS 외부 저장소에 마운트 되어 있으므로, 어떤 파드에 접속하더라도 사용자는 동일한 웹 화면과 최신 로그 데이터를 확인할 수 있습니다.

4.2.2. 데이터베이스 서버

물품 보관함의 '실시간 상태 동기화'를 위해 MySQL Master-Slave 복제를 구축합니다. 마스터는 모든 실시간 서비스 트래픽을 처리하고, 슬레이브는 크론탭 기반의 주기적인 백업과 데이터 조회를 분담합니다. 이를 통해 읽기/쓰기 성능을 최적화하고, 장애 발생 시에도 외부 NFS에 격리 저장된 스냅샷을 통해 즉각적인 복구가 가능한 고가용성 인프라를 지향합니다.



▼ DB (Master-Slave 복제)

구분	마스터 (Master) DB (172.16.0.8)	슬레이브 (Slave) DB (172.16.0.9)
핵심 역할	**실시간 서비스 운영 (Full Service)**	**데이터 조회 및 백업 (Backup Dedicated)**
주요 작업	모든 읽기/쓰기 요청 처리 (조회, 예약, 상태 변경 등)	`Crontab` 스케줄러를 통한 주기적 DB 덤프 및 전체 스냅샷 생성
데이터 동기화	변경 이력을 Binary Log에 기록 및 전송	Master의 로그를 Relay Log로 재현하여 실시간 동기화
저장 및 복구	실시간 트래픽 처리 및 DB 상태 유지	HostPath를 통해 클러스터 외부 NFS 스토리지에 `.sql` 격리 저장

▼ 1. DB 마스터-슬레이브 설정

```
MariaDB [(none)]> show master status;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000003 |      328 |              |                  |
+------------------+----------+--------------+------------------+
1 row in set (0.000 sec)
```

```
sy@db-s: ~
MariaDB [(none)]> CHANGE MASTER TO
    -> MASTER_HOST='172.16.0.8',
    -> MASTER_USER='repl_user',
    -> MASTER_PASSWORD='1234',
    -> MASTER_LOG_FILE='mysql-bin.000003',
    -> MASTER_LOG_POS=328;
Query OK, 0 rows affected, 1 warning (0.008 sec)
```

```
MariaDB [(none)]> SHOW SLAVE HOSTS;
+-----------+------------+------+-----------+
| Server_id | Host       | Port | Master_id |
+-----------+------------+------+-----------+
|         2 | 172.16.0.9 | 3306 |         1 |
+-----------+------------+------+-----------+
1 row in set (0.002 sec)
```

DB마스터에서 바이너리 파일을 통해 file번호와 position 값으로 슬레이브를 설정해줍니다.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: 172.16.0.8
                  Master_User: repl_user
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000003
          Read_Master_Log_Pos: 328
               Relay_Log_File: mysqld-relay-bin.000002
                Relay_Log_Pos: 555
        Relay_Master_Log_File: mysql-bin.000003
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
```

> 1. **Slave_IO_Running: Yes (로그 배달)**
> **:** Master의 **Binary Log**를 읽어오고 있는 상태인가?
> 2. **Slave_SQL_Running: Yes (쿼리 실행)**
> **:** 읽어온 로그를 **Relay Log**로 재현하는 상태인가?
>
> ⇒ **복제 프로세스 가동(Synchronization)**: 바이너리 로그가 슬레이브로 넘어가서 '저장'된 것이 릴레이 로그입니다.

▼ 2. Master DB: 백업 및 NFS 서버 전송

경로	역할	현재 상태
`/mnt/DATA2/master/data`	**Master용 백업**	`master_backup.sql` 보관 (초기용)
`/mnt/DATA2/slave/data`	**Slave용 백업**	복원용 덤프 보관

Master-Slave 복제(Replication) 설정 시, 데이터 동기화를 위해 Master에서 덤프 파일(스냅샷)을 생성하여 NFS 서버로 전송합니다. 이후 Slave 서버에서 NFS 마운트 상태를 확인하고, 해당 백업 파일을 활용해 데이터를 복원함으로써 초기 동기화 과정을 완료합니다.

```
# 데이터 백업 단계(mariadb-dump): SQL형태로 추출
sudo mariadb-dump -u root -p --all-databases \
--master-data=2 --single-transaction --routines \
--triggers > /tmp/master_backup.sql
```

```
# 데이터 전송 단계(scp): 외부 서버로 복사
scp /tmp/master_backup.sql sy@172.16.0.3:~/
```

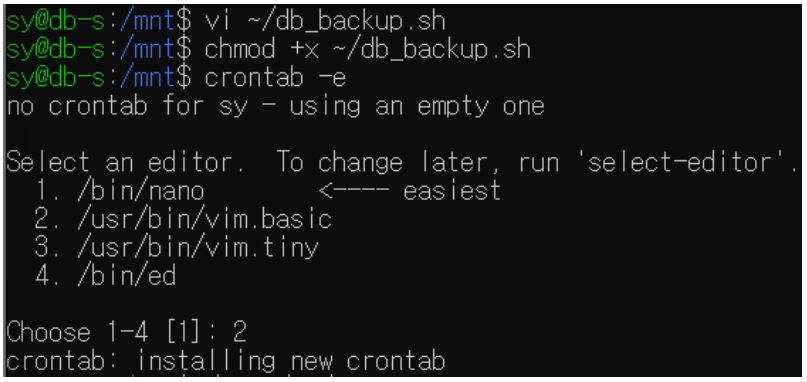
▼ **3. Slave DB: 주기적인 백업 파일 갱신 및 보관**

백업용 파일의 초기값을 불러왔으므로, 주기적인 slave 백업 파일의 보관을 위해 hostpath를 구현한 yaml 파일과 crontab(크론탭)을 활용해줍니다.

▼ **db-backup-hostpath.yaml**

```
# db-backup-hostpath.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backup-monitor
  namespace: locker-view-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backup-monitor
  template:
    metadata:
      labels:
        app: backup-monitor
    spec:
      containers:
      - name: monitor
        image: busybox
        command: ["sh", "-c", "while true; do ls -lh /backup-data; sleep
3600; done"]
        volumeMounts:
        - name: nfs-hostpath
          mountPath: /backup-data  # 파드 내부에서 접근할 경로
      volumes:
      - name: nfs-hostpath
        hostPath:
          path: /mnt/db_backup     # 워커 노드에 마운트한 바로 그 경로
          type: Directory
```

▼ **crontab 설정**



```
0 3 * * * /bin/bash /home/sy/db_backup.sh >> /home/sy/backup.log 2>&1
```

```
sy@db-s:/mnt$ ls -l /mnt/db_backup
total 2428
-rw-rw-r-- 1 share_user share_user 2485008 Mar 18 05:16 db_full_20260318_051625.sql
sy@db-s:/mnt$ head -n 50 /mnt/db_backup/db_full_20260318_051625.sql
/*M!999999\- enable the sandbox mode */
-- MariaDB dump 10.19  Distrib 10.11.14-MariaDB, for debian-linux-gnu (x86_64)
--
-- Host: localhost    Database:
-- ------------------------------------------------------
-- Server version       10.11.14-MariaDB-0ubuntu0.24.04.1-log

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
```

```
sy@nfs:/mnt/DATA2/master/data$ ls -la
total 2440
drwxrwxr-x 2 share_user share_user    4096 Mar 18 02:16 .
drwxr-xr-x 3 share_user share_user    4096 Mar 18 01:23 ..
-rw-rw-r-- 1 share_user share_user 2488048 Mar 18 02:16 master_backup.sql
```

백업 파일이 크론탭에 의해 저장되었고, 이제 nfs에서의 마운트 된 공유 디렉토리에서도 백업파일을 확인할 수 있습니다.

```
sy@db-s:/mnt$ df -hT | grep db_backup
172.16.0.3:/mnt/DATA2/slave/data  nfs4    25G  4.8M   24G   1% /mnt/db_backup
```

db-s의 /mnt/db_backup 폴더는 nfs의 공유 디렉토리와 마운트 되어있다.

```
sy@nfs:/mnt/DATA2/slave/data$ tail db_full_20260319_030001.sql

/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;
/*!40014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;
/*!40014 SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;

-- Dump completed on 2026-03-19  3:00:03
```

크론탭 설정에서 3시에 지정한 DB 백업 덤프 파일이 db-s와 마운트 된 nfs 공유 디렉토리로 전달 받은 것을 확인할 수 있습니다.

🔄 데이터 흐름(Data Flow)

1. **백업 생성 (Slave VM):** `db-s`의 크론탭이 `mysqldump`를 실행하여 `/mnt/db_backup`에 `.sql` 파일을 생성합니다.
2. **실시간 전송 (NFS):** 해당 폴더가 NFS 서버(`DATA2/slave/data`)와 마운트되어 있어, 파일 생성 즉시 외부 저장소로 실시간 저장됩니다.
3. **데이터 공유 (Worker Node):** k8s 워커 노드들도 동일한 NFS 경로를 마운트하고 있어, 슬레이브가 생성한 백업 파일을 즉시 공유받습니다.
4. **실시간 조회 (Pod):** 파일이 생성되자마자 NFS를 통해 워커 노드로 공유되고, 이미 연결된 `HostPath`를 통해 파드 안에서 즉시 최신 백업본을 읽을 수 있게 됩니다.

▼ 🚨 VM웨어 전원을 껐을 때 생기는 일

```
sy@db-s:/mnt/db_backup$ ls /mnt/db_backup
db_full_20260318_051625.sql  db_full_20260319_030001.sql
db_full_20260318_063513.sql  db_full_20260320_030001.sql
```

```
sy@nfs:/mnt/DATA2/slave/data$ ls
db_full_20260318_051625.sql  db_full_20260318_063513.sql  db_full_20260319_030001.sql
```

VM웨어의 전원을 껐을 때, db-s에 추가된 백업 덤프 파일에 마운트 된 nfs 공유 디렉토리에 전달되지 않습니다. 이때는 다시 umount-mount 과정을 거쳐서 갱신해줍니다.

```
sy@db-s:/mnt/db_backup$ cd ~
sy@db-s:~$ sudo umount /mnt/db_backup
sy@db-s:~$ ls -lh /mnt/db_backup
total 0
sy@db-s:~$ sudo mount -a
sy@db-s:~$ df -h | grep db_backup
172.16.0.3:/mnt/DATA2/slave/data    25G  9.5M   24G   1% /mnt/db_backup
```

하지만, 이 과정을 VM웨어를 껐을 때를 대비해서 db-s에 매번 접속해 거치기에는 번거로우므로 백업 스크립트에 관련 설정을 추가해줍니다.

⇒ 껐을 때의 환경은 마운트가 안 되어있으므로 마운트 체크 권한을 추가합니다. (+) share_user의 권한(644)도 수정했습니다.

▼ **db_backup.sh (추가)**

```
#!/bin/bash

# [추가] 크론탭 실행을 위한 환경 변수 설정
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# 1. 설정
BACKUP_DIR="/mnt/db_backup"
DATE=$(date +%Y%m%d_%H%M%S)
FILE_NAME="db_full_$DATE.sql"

# [추가] 마운트 체크: 마운트가 안 되어 있으면 로컬 디스크를 보호하기 위해 종료
if ! mountpoint -q "$BACKUP_DIR"; then
    echo "Error: $BACKUP_DIR is not mounted! Backup aborted to save loca
l disk space."
    exit 1
fi

# [추가] 권한 설정: 그룹에도 쓰기 권한(664)을 주기 위해 umask 설정
umask 002

# 2. 백업 실행 (비밀번호 1234)
# > 대신 -r (result) 옵션을 쓰거나 파이프를 쓰는 방식도 있지만, 현재 방식도 작동합
니다.
mysqldump -u root -p1234 --all-databases > "$BACKUP_DIR/$FILE_NAME"

# 3. 소유권 변경 (UID 1001:share_user)
# sudo를 쓰려면 crontab 실행 시 password 없는 sudo 권한이 필요할 수 있습니다.
sudo chown 1001:1001 "$BACKUP_DIR/$FILE_NAME"

# 4. 오래된 파일 삭제 (7일 경과)
find "$BACKUP_DIR" -type f -name "*.sql" -mtime +7 -delete

echo "Backup completed: $FILE_NAME"
```

```
sy@nfs:/mnt/DATA2/slave/data$ ls -l
total 12152
-rw-rw-r-- 1 share_user share_user 2485008 Mar 18 05:16 db_full_20260318_051625.sql
-rw-rw-r-- 1 share_user share_user 2485008 Mar 18 06:35 db_full_20260318_063513.sql
-rw-r--r-- 1 share_user share_user 2488777 Mar 19 03:00 db_full_20260319_030001.sql
-rw-r--r-- 1 share_user share_user 2489727 Mar 20 03:00 db_full_20260320_030001.sql
-rw-rw-r-- 1 share_user share_user 2489727 Mar 20 03:28 db_full_20260320_032824.sql
```

이제 권한에 맞춰서 마운트 된 nfs의 공유 디렉토리에 잘 전달 받은 것을 확인할 수 있습니다.

▼ DB 구성

▼ DB 생성 및 테이블 생성 후, 초기 데이터 삽입

< init.sql >

```
-- 1. 데이터베이스 생성 및 선택
CREATE DATABASE IF NOT EXISTS locker_system;
USE locker_system;

-- 2. 사물함 테이블 생성 (제약 조건 강화)
CREATE TABLE IF NOT EXISTS lockers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    station_name VARCHAR(20) NOT NULL,        -- 서울역, 부산역, 대구역
    locker_num INT NOT NULL,                  -- 각 역의 사물함 번호 (1~12)
    password VARCHAR(100) NOT NULL DEFAULT '0000', -- 초기값 0000, 예약 시 4자
리 필수
    phone VARCHAR(20),                        -- 예약자 연락처
    status ENUM('available', 'occupied') DEFAULT 'available',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMEST
AMP,

    -- [보안 강화] 비밀번호는 반드시 숫자 4자리여야 함
    CONSTRAINT chk_password_format CHECK (password REGEXP '^[0-9]{4}$'),

    -- [데이터 무결성] 한 역에 동일한 사물함 번호 중복 방지
    UNIQUE KEY unique_locker (station_name, locker_num)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. 초기 데이터 삽입 (3개 역 x 12개 = 총 36개)
-- 서울역 초기화
INSERT INTO lockers (station_name, locker_num)
SELECT '서울역', n FROM (
    SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT
5 UNION SELECT 6
    UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELE
CT 11 UNION SELECT 12
) AS nums;

-- 부산역 초기화
INSERT INTO lockers (station_name, locker_num)
SELECT '부산역', n FROM (
    SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT
5 UNION SELECT 6
    UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELE
CT 11 UNION SELECT 12
) AS nums;
```

```
-- 대구역 초기화
INSERT INTO lockers (station_name, locker_num)
SELECT '대구역', n FROM (
    SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT
5 UNION SELECT 6
    UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELE
CT 11 UNION SELECT 12
) AS nums;
```

< 구성 절차 >

```
sy@db-m: ~
MariaDB [(none)]> CREATE DATABASE IF NOT EXISTS locker_system;
Query OK, 1 row affected (0.027 sec)

MariaDB [(none)]> USE locker_system;
Database changed
MariaDB [locker_system]> CREATE TABLE IF NOT EXISTS lockers (
    -> id INT AUTO_INCREMENT PRIMARY KEY,
    -> station_name VARCHAR(20) NOT NULL,
    -> locker_num INT NOT NULL,
    -> password VARCHAR(100) NOT NULL DEFAULT '0000',
    -> phone VARCHAR(20),
    -> status ENUM('available', 'occupied') DEFAULT 'available',
    -> updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
    -> CONSTRAINT chk_password_format CHECK (password REGEXP '^[0-9]{4}$'),
    -> UNIQUE KEY unique_locker (station_name, locker_num)
    -> ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Query OK, 0 rows affected (0.118 sec)
```

locker_system 데이터베이스 생성 후, lockers 테이블 생성

```
sy@db-m: ~
MariaDB [locker_system]> INSERT INTO lockers (station_name, locker_num)
    -> SELECT '서울역', n FROM (
    -> SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT
 6
    -> UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELECT 11 UNION SEL
ECT 12
    -> ) AS nums;
Query OK, 12 rows affected (0.046 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

서울역 사물함 데이터 삽입

```
MariaDB [locker_system]> INSERT INTO lockers (station_name, locker_num)
    -> SELECT '부산역', n FROM (
    -> SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT
 6
    -> UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELECT 11 UNION SEL
ECT 12
    -> ) AS nums;
Query OK, 12 rows affected (0.003 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

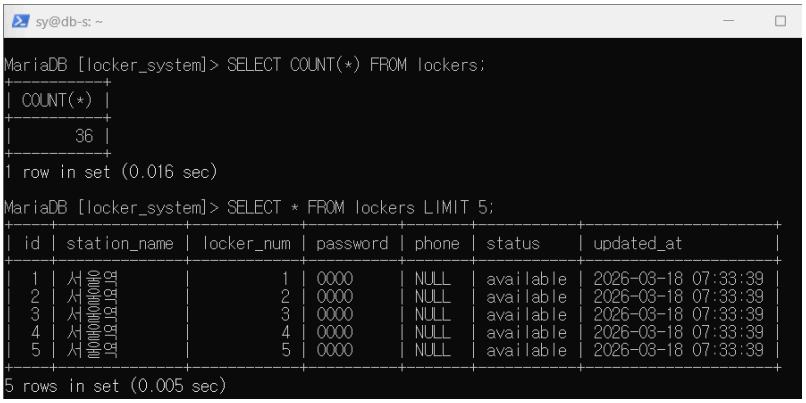
부산역 사물함 데이터 삽입

```
MariaDB [locker_system]> INSERT INTO lockers (station_name, locker_num)
    -> SELECT '대구역', n FROM (
    -> SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT
 6
    -> UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELECT 11 UNION SEL
ECT 12
    -> ) AS nums;
Query OK, 12 rows affected (0.009 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

대구역 사물함 데이터 삽입

컬럼명	타입	제약 조건	설명
id	`INT`	`PK`, `AUTO_INCREMENT`	각 사물함의 고유 식별 번호
station_name	`VARCHAR(20)`	`NOT NULL`	서비스 지역 (서울역, 부산역, 대구역)

locker_num	`INT`	`NOT NULL`, `UNIQUE(S, L)`	각 역 내 사물함 번호 (1~12), 역별 중복 방지
password	`VARCHAR(100)`	`DEFAULT '0000'`, `CHECK`	예약용 비밀번호 (초기값 '0000', 숫자 4자리 필수)
phone	`VARCHAR(20)`	`NULL`	예약자 연락처 정보
status	`ENUM`	`DEFAULT 'available'`	사물함 상태 (`available`, `occupied`)
updated_at	`TIMESTAMP`	`DEFAULT CURRENT`, `ON UPDATE`	최신 상태 변경 시 자동 갱신 시점



locker_systam이라는 데이터베이스에 lockers를 조회해보면 36개의 데이터가 들어가있는 것을 확인할 수 있고, 이 데이터들은 전부 status가 'available'인 초기값으로 예약시 'occupied'로 상태가 변한다.

▼ K8s-m와의 DB 연결 및 계정 분리

▼ mariadb-external.yaml

```yaml
# mariadb-external.yaml
apiVersion: v1
kind: Service
metadata:
  name: mariadb-master-svc  # FastAPI 코드(또는 Env)에서 사용할 DB 호스트 이름
  namespace: locker-view-ns
spec:
  ports:
    - port: 3306            # 서비스가 노출할 포트
      targetPort: 3306      # 실제 DB 서버의 포트
---
apiVersion: v1
kind: Endpoints
metadata:
  name: mariadb-master-svc  # Service 이름과 반드시 일치해야 함
  namespace: locker-view-ns
subsets:
  - addresses:
      - ip: 172.16.0.8      # MariaDB 마스터 서버의 실제 IP
```

```
    ports:
      - port: 3306
```

```
sy@k8s-m:~/project/k8s-manifests$ kubectl get endpoints -n locker-view-ns mariadb-master-svc
NAME                ENDPOINTS          AGE
mariadb-master-svc  172.16.0.8:3306    5s
```

mariadb 서비스에 엔드포인트 할당

계정명	주요 임무
`repl_user` **복제 전용 계정**	**DB 서버 간의 통신 전담:** 마스터(`db-m`)의 데이터를 슬레이브(`db-s`)가 그대로 복사해올 때만 사용합니다.
`sy_user` **서비스 접속 계정**	**애플리케이션 접속 전담:** 쿠버네티스의 FastAPI 파드들이 실제 데이터를 조회하거나 수정할 때 사용합니다.

4.2.3. 애플리케이션 서버

물품 보관함은 독립적인 기능을 수행하는 3개의 Fastapi Locker 서비스(`View`, `Reserve`, `Admin`)가 컨테이너로 배포되어 수행하며, 환경 변수를 통해 DB 접속 정보를 동적으로 관리합니다.

서비스명	역할 (Role)	주요 기능 및 특징
Locker-View (포트번호: 8001)	**조회 서비스 (Read)**	특정 역의 사물함 번호 및 실시간 예약 가능 여부(O/X) 조회
Locker-Reserve (포트번호: 8002)	**예약 서비스 (Write)**	사물함 선택 및 비밀번호 설정('0000' 초기 비번 사용 차단 로직 포함)
Locker-Admin (포트번호: 8003)	**관리 서비스 (Control)**	장기 방치 사물함 강제 초기화(Reset) 및 대시보드 조회 연동 (grafana.sy-locker-view.com)

1. **Locker-View (조회 서비스: Read 위주)**
   - **로드밸런싱 및 확장:** 시스템 효율을 위해 fastAPI 역할별로 분리 전략을 채택했습니다. Nginx를 통해 여러 개의 복제된 파드로 분산 처리됩니다. 덕분에 특정 시점에 사용자가 몰려도 전체 시스템에 영향을 주지 않는 가용성을 보장합니다.
   - **설정 관리:** 쿠버네티스 환경 변수(`env`)를 통해 DB 설정값을 주입받습니다. 특히 `mariadb-master-svc`라는 내부 서비스 이름을 호스트명으로 사용하여, 실제 DB 서버의 IP가 변경되더라도 할당된 서비스 엔드포인트를 통해 찾아갑니다.

2. **Locker-Reserve (예약 서비스: Write 위주)**
   - **데이터 무결성:** Nginx는 예약 요청을 예약 서비스 파드로 안내합니다. 이를 통해 동시 예약 상황에서도 데이터가 꼬이지 않도록 관리합니다.
   - **보안 계정 운영:** 애플리케이션 접속 시 최소 권한 원칙에 따라 서비스 전용 계정(`sy_user`)을 사용합니다. 이는 DB 서버 간의 대화에 쓰이는 복제 전용 계정(`repl_user`)과 엄격히 분리되어 있어, 만에 하나 발생할 수 있는 보안 사고의 범위를 최소화합니다.

3. **Locker-Admin (관리자 서비스: Control)**

- **게이트웨이 보안:** Nginx 게이트웨이를 통해 관리자 경로(`/admin/`)에 대한 강력한 접근 제어(ACL)를 수행합니다. 승인되지 않은 비인가 접근은 즉시 차단하고 전용 에러 페이지(`403.html`)로 안내하여 내부 인프라 구조를 보호합니다.

  ⛔ **ACL**
  1. `deny all;` 설정을 통해 "허용된 사람 외에는 일단 모두 도둑"으로 간주
  2. 관리자의 PC IP(`10.0.255.8`)와 특정 공인 IP만 통과
  3. 직접 만든 `403.html`을 보여줌으로써, 서버의 종류(OS 버전 등)나 상세한 에러 정보를 외부에 노출하지 않음

- **프로토콜 최적화:** 관리자의 보안 접속(`HTTPS/443`) 요청이 들어올 경우, 내부 통신 효율을 높이기 위해 자동으로 표준 규격인 `HTTP(80)`로 전환(301 Redirect)하여 데이터 처리의 일관성과 속도를 동시에 확보했습니다.

▼ 1. fastapi 구성

```
/home/sy/project/
├── service-locker/      # [View]
│   ├── main.py
│   ├── Dockerfile
│   └── requirements.txt
├── service-reserve/     # [Reserve]
│   ├── main.py
│   ├── Dockerfile
│   └── requirements.txt
└── service-admin/       # [Admin]
    ├── main.py
    ├── Dockerfile
    └── requirements.txt
```

설정 키	환경 변수	기본값
host	DB_HOST	mariadb-master-svc
user	DB_USER	sy_user
password	DB_PASS	1234
database	DB_NAME	locker_system

▼ 1. service-locker 구성 (main.py)

```
from fastapi import FastAPI, HTTPException
import mysql.connector
import os  # 환경변수를 읽기 위해 추가

app = FastAPI()

# 환경변수에서 정보를 가져오고, 없으면 기본값(Default)을 사용함
db_config = {
    'host': os.getenv('DB_HOST', 'mariadb-master-svc'),
    'user': os.getenv('DB_USER', 'sy_user'),
    'password': os.getenv('DB_PASS', '1234'),
    'database': os.getenv('DB_NAME', 'locker_system')
}

@app.get("/api/lockers/{station}")
```

```
async def get_lockers(station: str):
    try:
        # DB 연결 시도
        conn = mysql.connector.connect(**db_config)
        cursor = conn.cursor(dictionary=True)

        # 쿼리 실행
        query = "SELECT locker_num as id, status FROM lockers WHERE station
_name = %s ORDER BY locker_num"
        cursor.execute(query, (station,))
        lockers = cursor.fetchall()

        cursor.close()
        conn.close()
        return lockers
    except Exception as e:
        # 에러 발생 시 상세 메시지 반환
        raise HTTPException(status_code=500, detail=str(e))
```

▼ **2. service-reserve 구성 (main.py)**

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import mysql.connector
import os  # 환경변수를 읽기 위해 추가

app = FastAPI()

class Reservation(BaseModel):
    station: str
    locker_id: int
    password: str
    phone: str

# 쿠버네티스 환경변수에서 정보를 가져옴 (기본값 설정 포함)
db_config = {
    'host': os.getenv('DB_HOST', 'mariadb-master-svc'),
    'user': os.getenv('DB_USER', 'sy_user'),
    'password': os.getenv('DB_PASS', '1234'),
    'database': os.getenv('DB_NAME', 'locker_system')
}

@app.post("/api/reserve")
async def reserve_locker(res: Reservation):
    # [비밀번호 정책 반영] 0000 차단 로직
    if res.password == "0000":
        raise HTTPException(status_code=400, detail="0000은 초기 비밀번호이므로
사용할 수 없습니다.")

    try:
        # DB 연결 시도
        conn = mysql.connector.connect(**db_config)
        cursor = conn.cursor()
```

```
        # 1. 사물함 상태 업데이트
        # available 상태인 것만 occupied로 변경하는 로직
        query = """
            UPDATE lockers
            SET password = %s, phone = %s, status = 'occupied'
            WHERE station_name = %s AND locker_num = %s AND status = 'avail
able'
        """
        cursor.execute(query, (res.password, res.phone, res.station, res.lo
cker_id))

        # 업데이트된 행이 0개라면 (이미 예약됐거나 정보가 틀린 경우)
        if cursor.rowcount == 0:
            cursor.close()
            conn.close()
            raise HTTPException(status_code=400, detail="이미 예약되었거나 존재
하지 않는 사물함입니다.")

        conn.commit()
        cursor.close()
        conn.close()
        return {"message": "success"}
    except Exception as e:
        # 에러 발생 시 상세 메시지 반환
        raise HTTPException(status_code=500, detail=str(e))
```

▼ **3. service-admin 구성 (main.py)**

```
from fastapi import FastAPI, HTTPException
import mysql.connector
from pydantic import BaseModel
import os  # 환경변수를 읽기 위해 추가

app = FastAPI()

# 쿠버네티스 환경변수에서 DB 접속 정보를 동적으로 가져옵니다.
db_config = {
    'host': os.getenv('DB_HOST', 'mariadb-master-svc'),
    'user': os.getenv('DB_USER', 'sy_user'),
    'password': os.getenv('DB_PASS', '1234'),
    'database': os.getenv('DB_NAME', 'locker_system')
}

# 1. 전체 사물함 상세 현황 조회 (비밀번호, 전화번호 포함)
@app.get("/api/admin/lockers")
async def get_admin_lockers():
    try:
        # DB 연결 시도
        conn = mysql.connector.connect(**db_config)
        cursor = conn.cursor(dictionary=True)

        # 관리자용이므로 모든 컬럼을 조회합니다.
```

```
        query = "SELECT * FROM lockers ORDER BY station_name, locker_num"
        cursor.execute(query)
        result = cursor.fetchall()

        cursor.close()
        conn.close()
        return result
    except Exception as e:
        # 에러 발생 시 상세 메시지 반환 (디버깅용)
        raise HTTPException(status_code=500, detail=str(e))

# 2. 특정 사물함 강제 초기화 (비우기)
@app.post("/api/admin/reset/{locker_id}")
async def reset_locker(locker_id: int):
    try:
        conn = mysql.connector.connect(**db_config)
        cursor = conn.cursor()

        # 해당 사물함을 초기 상태(0000, NULL, available)로 되돌림
        query = """
            UPDATE lockers
            SET password = '0000', phone = NULL, status = 'available'
            WHERE id = %s
        """
        cursor.execute(query, (locker_id,))

        conn.commit()
        cursor.close()
        conn.close()
        return {"message": f"Locker {locker_id} has been reset."}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

▼ **4. fatapi 포트번호 별 Dockerfile 생성**

서비스명	역할	포트
service-locker	일반인용 현황 조회	8001
service-reserve	비회원 예약 신청	8002
service-admin	관리자용 제어/초기화	8003

```
FROM python:3.9-slim
WORKDIR /app

# 환경 변수 설정
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1

COPY requirements.txt .
RUN pip install --no-cache-dir fastapi uvicorn mysql-connector-python

COPY . .
```

```
EXPOSE 8003
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8003"]
```

▼ **5. Git을 통한 이미지 동기화**

```
sy@k8s-m:~/project$ git push -u origin main
Username for 'https://github.com': ParkSoyoung77
Password for 'https://ParkSoyoung77@github.com':
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 2 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (19/19), 6.45 KiB | 1.61 MiB/s, done.
Total 19 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/ParkSoyoung77/locker-project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
sy@k8s-m:~/project$
```

k8s-m에서 git push

```
sy@node1:~$ git clone https://github.com/ParkSoyoung77/locker-project.git ~/project
Cloning into '/home/sy/project'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 19 (delta 2), reused 19 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (19/19), 6.45 KiB | 943.00 KiB/s, done.
Resolving deltas: 100% (2/2), done.
sy@node1:~$ cd ~/project
sy@node1:~/project$ git pull origin main
From https://github.com/ParkSoyoung77/locker-project
 * branch            main       -> FETCH_HEAD
Already up to date.
```

git clone-git pull로 이미지 가져온 후, Docker 이미지 빌드

```
sy@k8s-m: ~/project
sy@k8s-m:~/project$ sudo docker images | grep sy-locker
sy-locker-admin     v1         5023dd7bc5a9   18 minutes ago   232MB
sy-locker-reserve   v1         2c55bb352154   18 minutes ago   232MB
sy-locker-view      v1         9ae363a8ddeb   18 minutes ago   232MB
sy@k8s-m:~/project$ sudo docker run --rm sy-locker-view:v1 ls -R /app
/app:
Dockerfile
main.py
requirements.txt
sy@k8s-m:~/project$ sudo docker run --rm sy-locker-reserve:v1 ls -R /app
/app:
Dockerfile
main.py
requirements.txt
sy@k8s-m:~/project$ sudo docker run --rm sy-locker-admin:v1 ls -R /app
/app:
Dockerfile
main.py
requirements.txt
```

각 워커노드에서 git clone-git pull로 이미지 가져오기

```
sy@node1: ~/project/service-admin
sy@node1:~/project/service-admin$ sudo docker images | grep sy-locker
sy-locker-admin     v1         32430a464ed8   2 minutes ago   232MB
sy-locker-reserve   v1         2475cbb96e87   2 minutes ago   232MB
sy-locker-view      v1         73bab8de455f   3 minutes ago   232MB
```

```
sy@node2: ~/project/service-admin
sy@node2:~/project/service-admin$ sudo docker images | grep sy-locker
sy-locker-admin     v1         97e95ec8cf46   2 minutes ago   232MB
sy-locker-reserve   v1         f38b24321902   2 minutes ago   232MB
sy-locker-view      v1         5a9430c9468d   4 minutes ago   232MB
```

k8s-m와 node1, node2에 도커 이미지가 잘 빌드된 것을 확인할 수 있습니다.

▼ **6. fastapi-services.yaml 배포**

```
# fastapi-services.yaml
# 1. 조회 서비스 (Locker)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: locker-view-deploy
  namespace: locker-view-ns
spec:
  replicas: 2
  selector:
    matchLabels:
      app: locker-view
  template:
    metadata:
      labels:
        app: locker-view
    spec:
      containers:
      - name: locker-app
        image: sy-locker-view:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 8001
        env:
        - name: DB_HOST
          value: "mariadb-master-svc"  # mariadb-external.yaml에서 만든 서비스
이름
        - name: DB_PORT
          value: "3306"
        - name: DB_USER
          value: "sy_user"
        - name: DB_PASS
          value: "1234"
        - name: DB_NAME
          value: "locker_system"
---
apiVersion: v1
kind: Service
metadata:
  name: locker-service-svc
  namespace: locker-view-ns
spec:
  selector:
    app: locker-view
  ports:
  - port: 8001
    targetPort: 8001
---
# 2. 예약 서비스 (Reserve)
apiVersion: apps/v1
```

```yaml
kind: Deployment
metadata:
  name: locker-reserve-deploy
  namespace: locker-view-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: locker-reserve
  template:
    metadata:
      labels:
        app: locker-reserve
    spec:
      containers:
      - name: reserve-app
        image: sy-locker-reserve:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 8002
        env:
        - name: DB_HOST
          value: "mariadb-master-svc"  # mariadb-external.yaml에서 만든 서비스
이름
        - name: DB_PORT
          value: "3306"
        - name: DB_USER
          value: "sy_user"
        - name: DB_PASS
          value: "1234"
        - name: DB_NAME
          value: "locker_system"
---
apiVersion: v1
kind: Service
metadata:
  name: reserve-service-svc
  namespace: locker-view-ns
spec:
  selector:
    app: locker-reserve
  ports:
  - port: 8002
    targetPort: 8002
---
# 3. 관리 서비스 (Admin)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: locker-admin-deploy
  namespace: locker-view-ns
spec:
  replicas: 1
```

```yaml
  selector:
    matchLabels:
      app: locker-admin
  template:
    metadata:
      labels:
        app: locker-admin
    spec:
      containers:
      - name: admin-app
        image: sy-locker-admin:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 8003
        env:
        - name: DB_HOST
          value: "mariadb-master-svc"
        - name: DB_PORT
          value: "3306"
        - name: DB_USER
          value: "sy_user"
        - name: DB_PASS
          value: "1234"
        - name: DB_NAME
          value: "locker_system"
---
apiVersion: v1
kind: Service
metadata:
  name: admin-service-svc
  namespace: locker-view-ns
spec:
  selector:
    app: locker-admin
  ports:
  - port: 8003
    targetPort: 8003
```

```
sy@k8s-m:~/project/k8s-manifests$ kubectl apply -f fastapi-services.yaml
deployment.apps/locker-view-deploy created
service/locker-service-svc created
deployment.apps/locker-reserve-deploy created
service/reserve-service-svc created
deployment.apps/locker-admin-deploy created
service/admin-service-svc created
sy@k8s-m:~/project/k8s-manifests$ kubectl apply -f nginx-conf.yaml
configmap/nginx-config configured
sy@k8s-m:~/project/k8s-manifests$ kubectl rollout restart deployment nginx-deploy-lab -n lock
er-view-ns
deployment.apps/nginx-deploy-lab restarted
```

fastapi-services.yaml 배포

▼ 7. 로컬의 C:\Windows\System32\drivers\etc\host 파일 설정

```
10.0.105.1  sy-locker-view.com
10.0.105.1  www.sy-locker-view.com
```

```
10.0.105.1  grafana.sy-locker-view.com
```

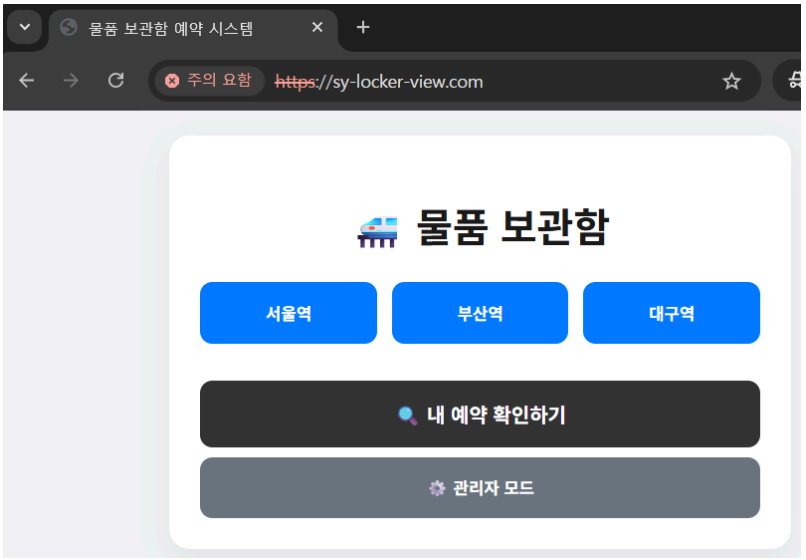
```
sy@k8s-m:~/project$ kubectl get pods -n locker-view-ns
NAME                                  READY   STATUS    RESTARTS        AGE
backup-monitor-65c65f5d97-kjdt5       1/1     Running   1 (3d1h ago)    3d20h
locker-admin-deploy-98887cb48-5vvsj   1/1     Running   1 (3d1h ago)    3d17h
locker-reserve-deploy-7c5c5f9b4-7md9c 1/1     Running   1 (3d1h ago)    3d16h
locker-view-deploy-5f78ff6557-2jsvx   1/1     Running   1 (3d1h ago)    3d17h
locker-view-deploy-5f78ff6557-hnmp8   1/1     Running   1 (3d1h ago)    3d16h
nginx-deploy-lab-66d97945dd-9dmhp     1/1     Running   0               2d18h
nginx-deploy-lab-66d97945dd-mdjnh     1/1     Running   0               2d18h
nginx-deploy-lab-66d97945dd-pndc9     1/1     Running   0               2d18h
```

- **backup-monitor:** 워커 노드에 마운트된 HostPath를 통해 MariaDB 스냅샷 데이터를 NFS로 전송하며 실시간 백업 상태를 감시합니다.
- **locker-view-deploy:** 사용자가 각 역의 사물함 현황을 실시간으로 조회할 수 있도록 Read 전용 API를 제공합니다.
- **locker-reserve-deploy:** 사용자가 사물함을 선택하고 비밀번호를 설정하여 예약을 확정하는 Write 서비스를 처리합니다.
- **locker-admin-deploy:** 관리자가 사물함을 강제로 초기화하거나 전체 시스템 현황을 제어하는 관리 기능을 수행합니다.
- **nginx-deploy-lab:** 외부 요청을 각 서비스로 분산(Proxy)하고, PV-PVC로 할당된 nfs에서 메인 웹 페이지의 정적 자산을 사용자에게 서빙합니다.

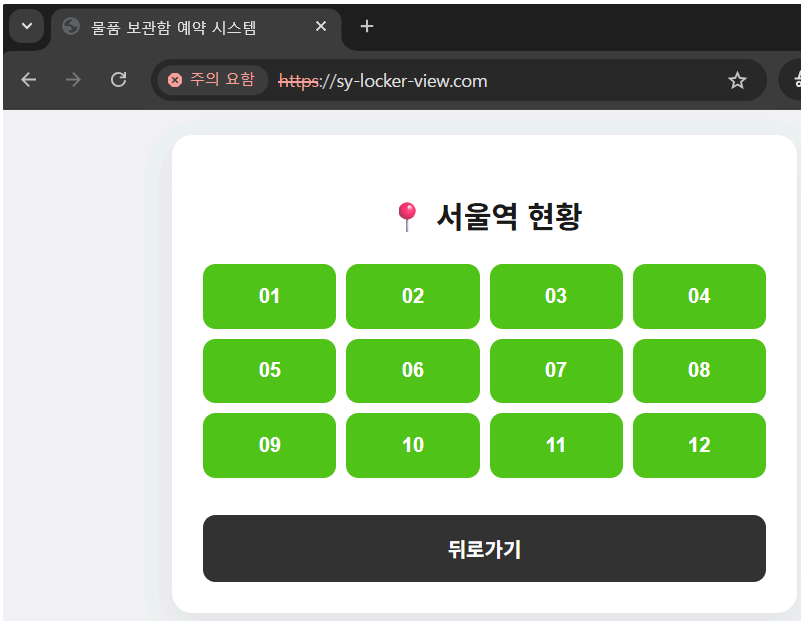
▼ 2. 웹 페이지 구현

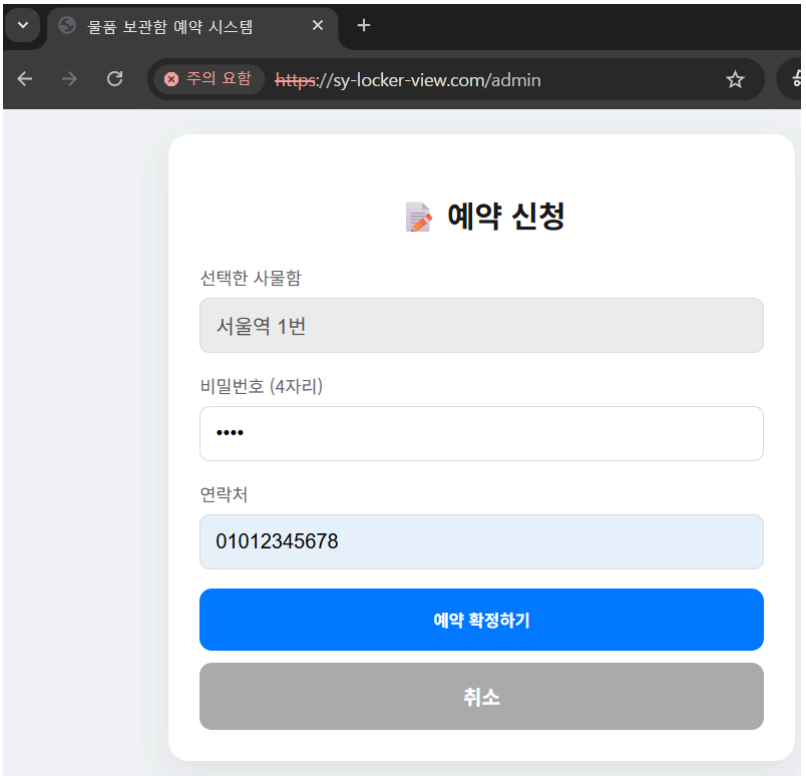
> **도메인 주소 (172.16.0.110)**
> http://sy-locker-view.com/

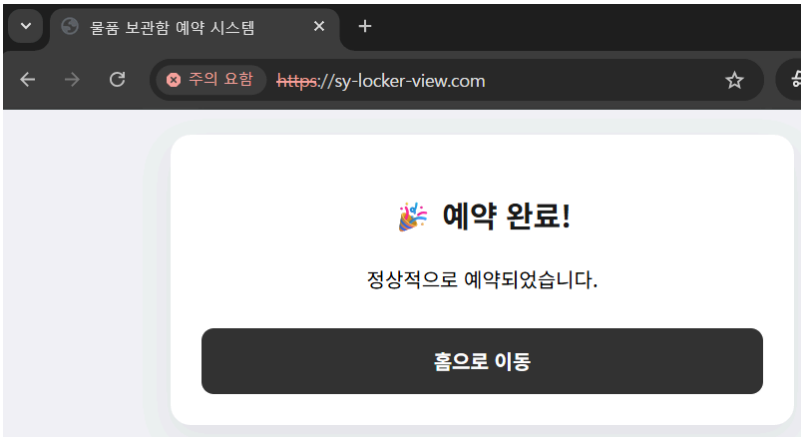
▼ 메인 홈페이지

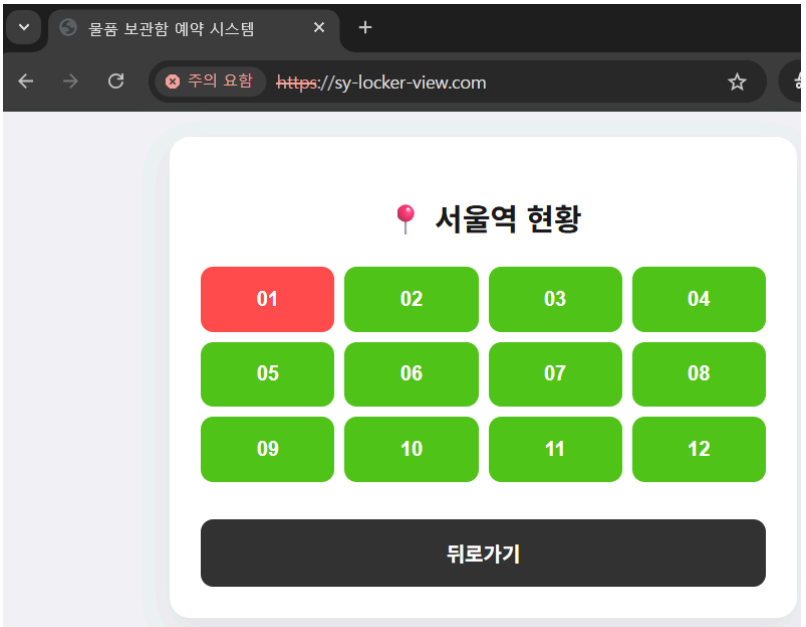


▼ 각 역 선택 + DB


물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com
서울역 현황
01
02
03
04
05
06
07
08
09
10
11
12
뒤로가기

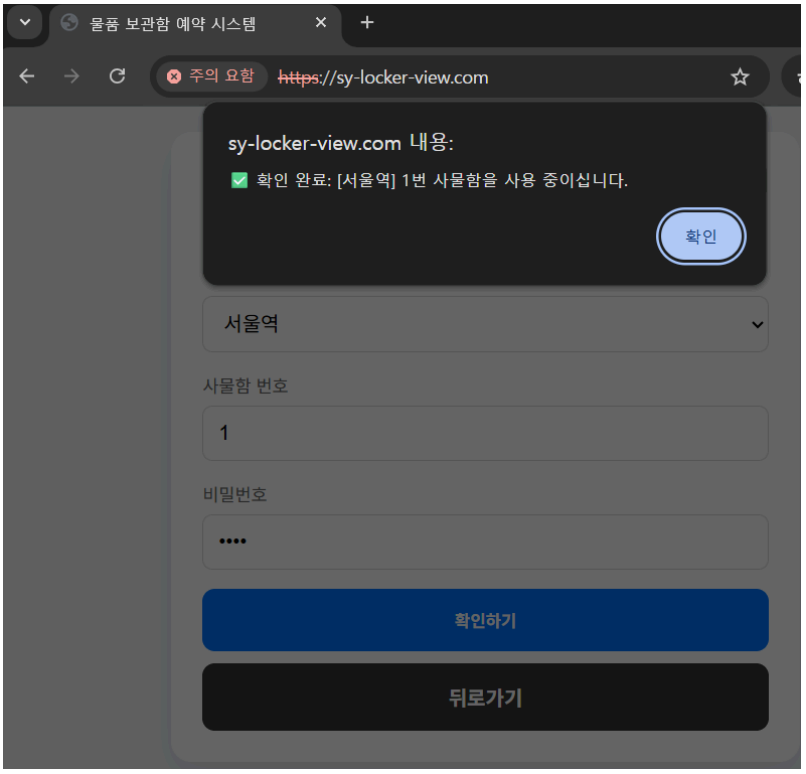

물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com/admin
예약 신청
선택한 사물함
서울역 1번
비밀번호 (4자리)
••••
연락처
01012345678
예약 확정하기
취소

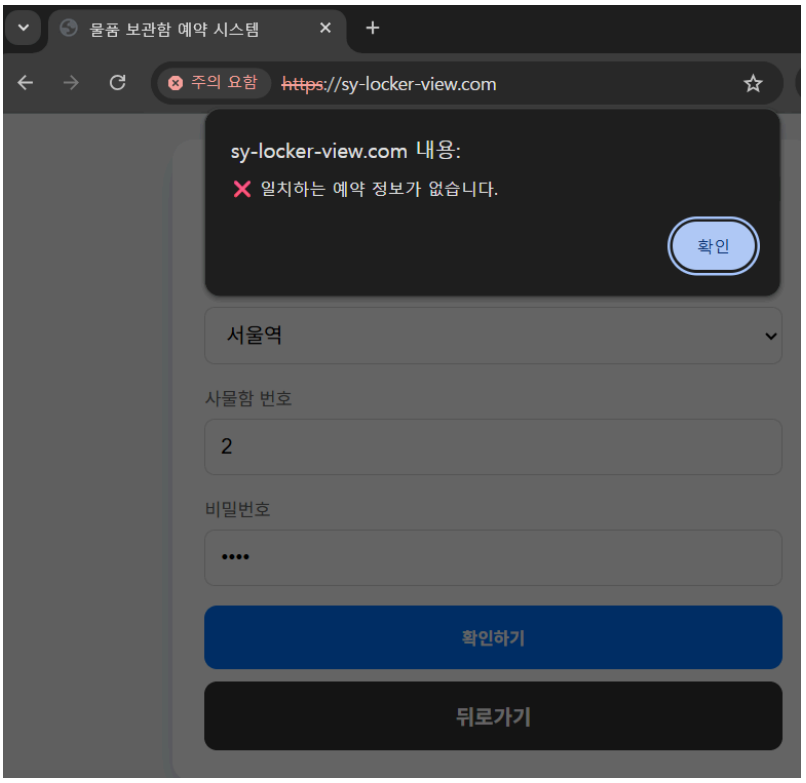




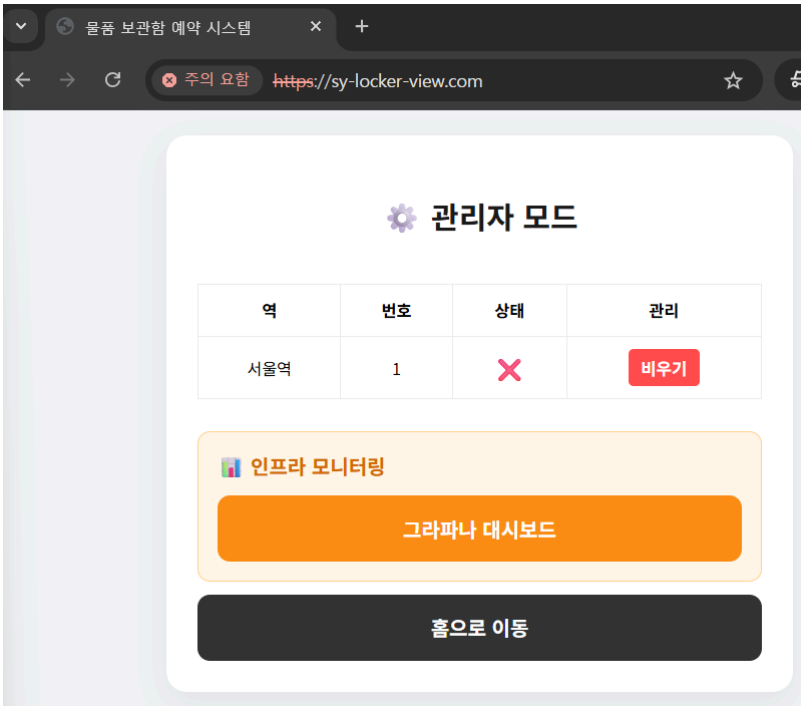

```
MariaDB [locker_system]> select * from lockers;
+----+--------------+------------+----------+-------------+-----------+---------------------+
| id | station_name | locker_num | password | phone       | status    | updated_at          |
+----+--------------+------------+----------+-------------+-----------+---------------------+
|  1 | 서울역       |          1 | 1234     | 01012345678 | occupied  | 2026-03-19 08:54:31 |
|  2 | 서울역       |          2 | 0000     | NULL        | available | 2026-03-18 07:33:39 |
|  3 | 서울역       |          3 | 0000     | NULL        | available | 2026-03-18 07:33:39 |
```

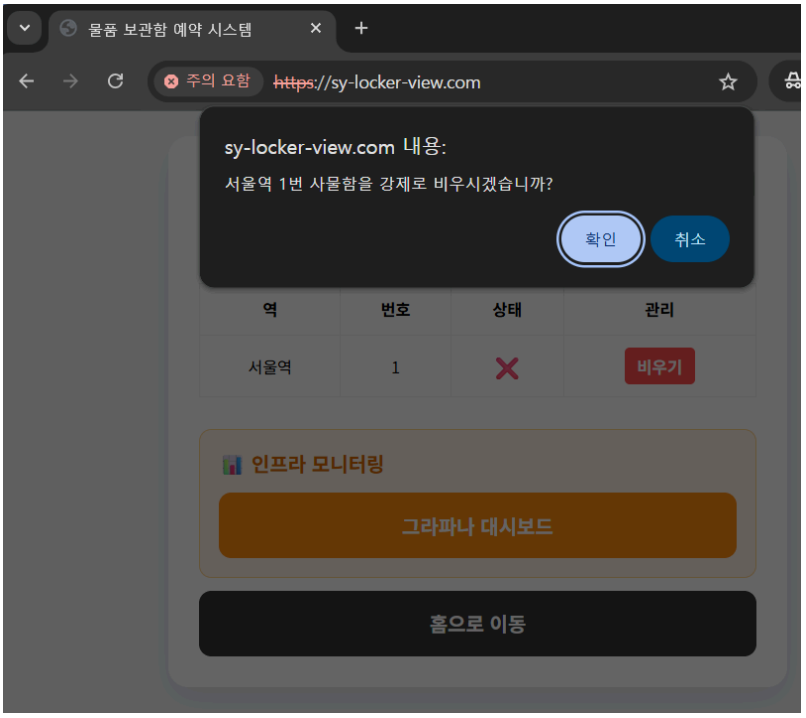
▼ 내 예약 확인하기

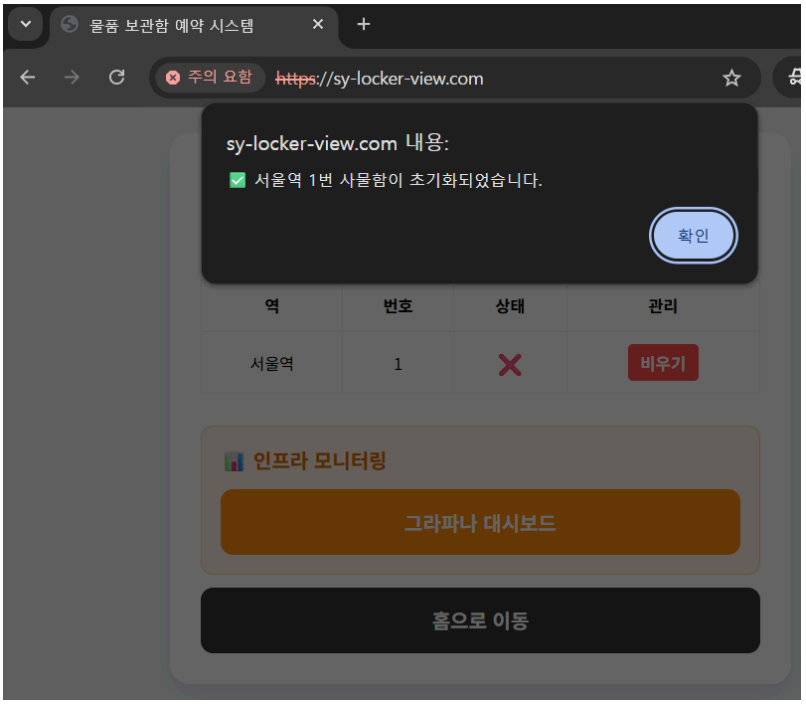

물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com
sy-locker-view.com 내용:
확인 완료: [서울역] 1번 사물함을 사용 중이십니다.
확인
서울역
사물함 번호
1
비밀번호
••••
확인하기
뒤로가기


물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com
sy-locker-view.com 내용:
일치하는 예약 정보가 없습니다.
확인
서울역
사물함 번호
2
비밀번호
••••
확인하기
뒤로가기

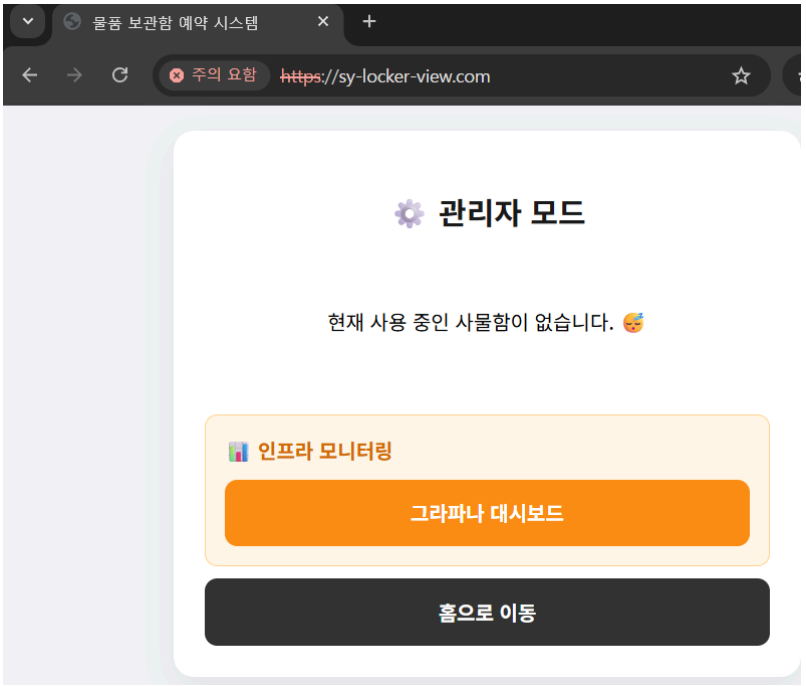
▼ 관리자 모드 + DB


물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com
관리자 모드
역
번호
상태
관리
서울역
1
비우기
인프라 모니터링
그라파나 대시보드
홈으로 이동

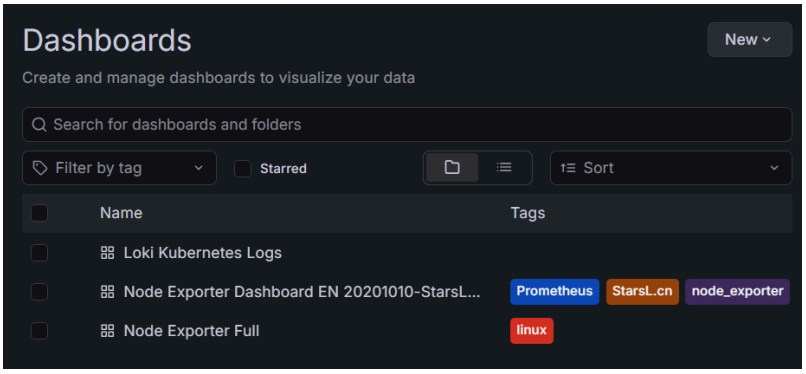

물품 보관함 예약 시스템
주의 요함
https://sy-locker-view.com
sy-locker-view.com 내용:
서울역 1번 사물함을 강제로 비우시겠습니까?
확인
취소
역
번호
상태
관리
서울역
1
비우기
인프라 모니터링
그라파나 대시보드
홈으로 이동



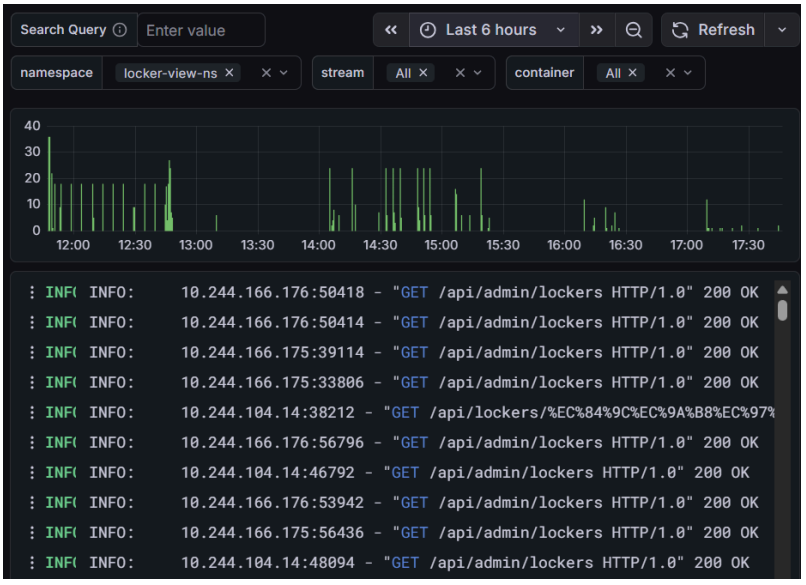

```
MariaDB [locker_system]> select * from lockers;
+----+--------------+-----------+----------+-------+-----------+---------------------+
| id | station_name | locker_num | password | phone | status    | updated_at          |
+----+--------------+-----------+----------+-------+-----------+---------------------+
|  1 | 서울역       |         1 | 0000     | NULL  | available | 2026-03-19 08:55:36 |
|  2 | 서울역       |         2 | 0000     | NULL  | available | 2026-03-18 07:33:39 |
|  3 | 서울역       |         3 | 0000     | NULL  | available | 2026-03-18 07:33:39 |
```



▼ 3. 그라파나 대시보드



▼ **통합 로그 모니터링 (Loki Kubernetes Logs)**

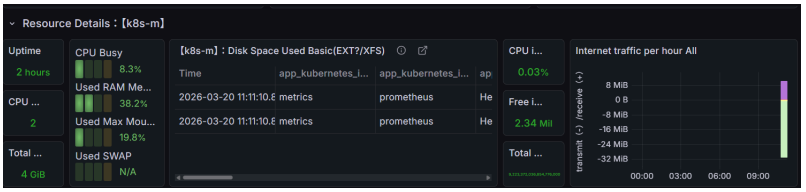


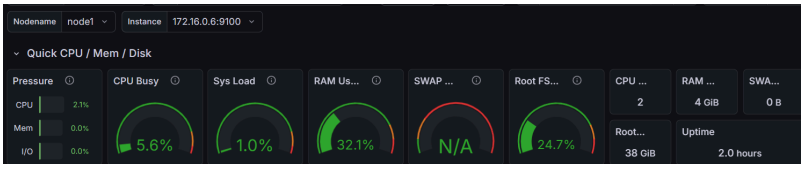
Grafana에서 Loki를 통해 열람한 로그

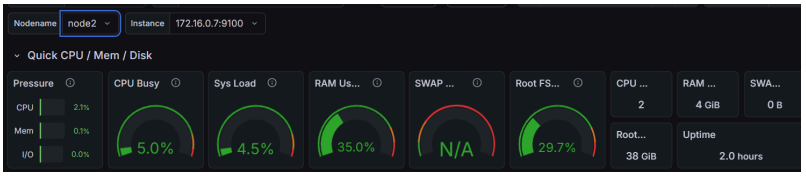
- **실시간 트래픽 그래프:** 상단의 차트를 통해 특정 시간대에 API 요청이 얼마나 집중되었는지 한눈에 파악 가능합니다.
- **컨테이너 로그 통합:** `locker-view-ns` 네임스페이스 내의 여러 파드에서 발생하는 로그가 로키(Loki)를 통해 중앙으로 잘 수집되고 있습니다.
- **상태 코드:** 로그 리스트 우측에 `200 OK`가 나열되어 있어, 백엔드 서비스들이 요청을 지연 없이 처리하고 있습니다.

▼ **프로메테우스 대시보드 (Node Exporter)**

각 노드(서버)의 CPU, 메모리, 디스크 같은 하드웨어 상태 데이터를 수집하여 프로메테우스를 통해 대시보드에 실시간으로 시각화







▼ 4. 로그값 분석

▼ 각각 VM의 nginx 로그(access.log/error.log) 열람

Bastion Host에서 조회한 nginx의 access.log: 최초로 도달하는 로그

nfs(/mnt/DATA1/nginx-logs)에서 조회한 nginx의 sy_access.log: fastapi의 default.conf를 거쳐 오는 로그

```
sy@nfs:/mnt/DATA1/nginx-logs$ tail -f sy_error.log
2026/03/18 03:50:51 [error] 21#21: *2 open() "/usr/share/nginx/html/api/lockers/부산역" faile
d (2: No such file or directory), client: 10.244.166.151, server: sy-locker-view.com, request
: "GET /api/lockers/%EB%B6%80%EC%82%B0%EC%97%AD HTTP/1.1", host: "sy-locker-view.com", referr
er: "https://sy-locker-view.com/"
2026/03/19 01:36:30 [error] 22#22: *3 open() "/usr/share/nginx/html/favicon.ico" failed (2: N
o such file or directory), client: 10.244.104.18, server: sy-locker-view.com, request: "GET /
favicon.ico HTTP/1.1", host: "sy-locker-view.com", referrer: "https://sy-locker-view.com/"
2026/03/19 01:39:01 [error] 22#22: *4 open() "/usr/share/nginx/html/favicon.ico" failed (2: N
o such file or directory), client: 10.244.104.18, server: sy-locker-view.com, request: "GET /
favicon.ico HTTP/1.1", host: "sy-locker-view.com", referrer: "https://sy-locker-view.com/"
2026/03/19 07:40:33 [error] 21#21: *28 access forbidden by rule, client: 10.244.104.55, serve
r: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:40:41 [error] 22#22: *27 access forbidden by rule, client: 10.244.104.55, serve
r: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:41:00 [error] 21#21: *24 access forbidden by rule, client: 10.244.104.55, serve
r: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:47:03 [error] 22#22: *1 access forbidden by rule, client: 10.244.104.55, server
: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:47:01 [error] 21#21: *1 access forbidden by rule, client: 10.244.104.55, server
: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:47:17 [error] 22#22: *1 access forbidden by rule, client: 10.244.104.55, server
: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
2026/03/19 07:47:22 [error] 22#22: *1 access forbidden by rule, client: 10.244.104.55, server
: sy-locker-view.com, request: "GET /admin/ HTTP/1.1", host: "sy-locker-view.com"
```

nfs(/mnt/DATA1/nginx-logs)에서 조회한 nginx의 sy_error.log: fastapi의 default.conf를 거쳐 오는 로그

1. **외부 진입 로그 (Bastion Nginx)**
   - **원본 IP 확인:** 모든 요청의 출발지가 `10.0.255.8`로 찍혀 있습니다. 이는 관리자의 실제 PC IP가 배스천 서버에 성공적으로 도달했음을 의미합니다. (DHCP 서버는 `10.0.255.1`~`10.0.255.254` 사이의 주소 중 하나를 접속하는 기기에 자동으로 부여합니다.)
   - **요청 다양성:** 메인 페이지(`/`), 사물함 조회(`/api/lockers/`), 예약(`/api/reserve`), 관리자 초기화(`/api/admin/`) 등 모든 API가 정상적으로 유입되고 있습니다.
   - **응답 코드:** 대부분 `200 OK`로, 네트워크 연결에 문제가 없습니다.

2. **내부 전송 및 보안 로그 (Nginx Ingress & Error)**
   - **실제 IP 전달:** `sy_access.log`를 보면 `10.0.255.8` 뒤에 `172.16.0.2`가 붙어 있습니다. 배스천이 원본 IP를 헤더에 담아 인그레스로 잘 넘겨주고 있다는 증거입니다.
   - **접근 차단 (403 Forbidden):** `sy_error.log`에서 `access forbidden by rule` 메시지가 다수 발견됩니다. 이는 관리자 경로(`/admin/`)에 대해 설정하신 **IP 필터링 정책**이 침입 시도를 성공적으로 막아내고 있음을 보여줍니다.

▼ 4.3. 네트워크 설계

4.3.1. 서비스 구성

```
[외부 사용자]
      ↓ (HTTP/HTTPS 접속: sy-locker-view.com)
[Bastion Nginx 게이트웨이] ──→ [관리자 ACL(10.0.255.8 허용)]
      ↓ (Reverse Proxy: 172.16.0.110 전달)
[K8s MetalLB & Ingress]
      ↓ (L7 라우팅: 서비스별 경로 분기)
[Nginx Pod (Frontend)] ↔ [FastAPI Pod (Backend)]
      ↑                        ↓ (Write)          ↓ (Read)
      │                [MariaDB Master] ──→ [MariaDB Slave]
      │                              (Replication)    ↓
      │                                      (Crontab + Dump)
      │                                              ↓
      └──────────── [ NFS Server ] ──────── [ /mnt/DATA2 ]
      (NFS PV/PVC)  /mnt/DATA1 (Logs)    /mnt/DATA2 (Web/DB)
```

▼ 네트워크 서비스 흐름

1. **접속 단계 (External Access)**

   [외부 사용자] → 도메인(sy-locker-view.com) 접속 → [Bastion Nginx 게이트웨이]

   - 역할: SSL 인증 및 관리자 IP 필터링(ACL) 수행

2. **클러스터 진입 (K8s Ingress)**

   [Bastion Nginx] → [MetalLB VIP: 172.16.0.110] → [Nginx Ingress Controller]

   - 역할: L7 레이어에서 도메인 및 경로(/api/ 등)분석 후 서비스 안내

3. **마이크로서비스 처리 (FastAPI Pods)**

   [Ingress] → [FastAPI: Locker-View/Reserve/Admin]

   - 역할: Fastapi에 할당된 해당 네임스페이스의 벡엔드르를 전용 파드로 라우팅

4. **데이터 저장 및 복제 (Persistence & Replication)**

   [FastAPI] → [MariaDB Master] → [MariaDB Slave]

   - 역할: Slave DB에서 크롭탭을 이용한 Dump 수행 후 NFS(/mnt/DATA2)에 저장

5. **공유 자원 활용 (NFS Storage)**

   [Nginx Pod] ↔ [NFS Server] (정적 웹 자원 서빙 및 통합 로그 기록)

   - 역할: /mnt/DATA1(로그), /mnt/DATA2(웹 소스 및 DB 백업)

▼ Bastion Host에서 외부 IP 받을 수 있도록 설정

모든 외부 트래픽은 **배스천 호스트(Bastion Host)의 Nginx**를 거쳐 내부망으로 전달되도록 설계하기 위해, 설정값들을 nginx에 적용하여 외부IP로 받아오게 합니다.

1. Bastion Host에서 파일을 수정하기 전에 K8s-M에서 선행 작업을 해줍니다. 명령어를 통해 Ingress ConfigMap을 설정해줍니다.

   ▼ < Ingress CofigMap 설정 >

```
# 1. 배스천 호스트(172.16.0.0/24)
# 를 신뢰할 수 있는 프록시로 등록
kubectl patch configmap ingress-nginx-controller -n ingress-nginx --type
merge -p '{"data":{"compute-full-forwarded-for":"true","use-forwarded-he
aders":"true","proxy-real-ip-cidr":"172.16.0.0/24","set-real-ip-from":"1
72.16.0.1, 172.16.0.0/24"}}'
```

```
# 2. 서비스 패치
# 원본 사용자 IP를 그대로 유지하도록 강제
kubectl patch svc ingress-nginx-controller -n ingress-nginx -p '{"spec":
{"externalTrafficPolicy":"Local"}}'
```

```
# 3. 메인 서비스 접속
# HTTP 요청->HTTPS로 자동 리다이렉트 기능 비활성화
kubectl patch ingress lab-ingress -n locker-view-ns --type merge -p '{"m
etadata": {"annotations": {"nginx.ingress.kubernetes.io/ssl-redirect":
"false", "nginx.ingress.kubernetes.io/force-ssl-redirect": "false"}}}'
```

```
# 4. 그라파나 대시보드
# 강제적인 SSL 리다이렉트를 막기
kubectl patch ingress grafana-ingress -n locker-view-ns --type merge -p
'{"metadata": {"annotations": {"nginx.ingress.kubernetes.io/ssl-redirec
t": "false", "nginx.ingress.kubernetes.io/force-ssl-redirect": "fals
e"}}}'
```

2. Bastion Host의 confing 파일들을 수정합니다.

▼ **/etc/keepalived/keepalived.conf**

```
# 1. Nginx 상태 체크 스크립트 정의 (instance 블록 바깥에 위치)
vrrp_script check_nginx {
    # 1. 쉘(-c)을 통해 전체 경로(/usr/bin/pidof)로 실행하도록 강제합니다.
    # 2. 뒤에 > /dev/null 을 붙여 출력 노이즈를 방지합니다.
    script "/bin/sh -c '/usr/bin/pidof nginx > /dev/null 2>&1'"
    interval 2
    weight -20
}

vrrp_instance VI_1 {
    state MASTER          # 여기는 MASTER
    interface ens37       # Host-only 인터페이스 이름 확인
    virtual_router_id 61
    priority 100          # Master는 Slave(90)보다 높게
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1234
    }
    virtual_ipaddress {
        172.16.0.100      # 서비스용 VIP
        172.16.0.200      # 관리용 VIP
    }

    # 2. 위에서 만든 스크립트 적용
    track_script {
        check_nginx
    }
```

```
}
```

▼ **/etc/nginx/sites-available/default.conf**

```
server {
        listen 80 default_server;
        listen [::]:80 default_server;

        server_name sy-locker-view.com www.sy-locker-view.com localhost
grafana.sy-locker-view.com;

        root /var/www/html;


        index index.html index.htm index.nginx-debian.html;

        location / {
            # 쿠버네티스 MetalLB(인그레스) IP로 전달
            proxy_pass http://172.16.0.110/;

            # [핵심] 원본 IP를 전달하는 헤더 3인방
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

            # 프록시 관련 추가 설정 (안정성)
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection 'upgrade';
            proxy_cache_bypass $http_upgrade;


            # 타임아웃 방지
            proxy_connect_timeout 90;
            proxy_send_timeout 90;
            proxy_read_timeout 90;

        }

        # 3. 관리자 전용 경로 (IP 제한 및 커스텀 403 적용)
        location /admin/ {
            # [핵심] 오늘 확인한 SY님 PC의 실제 IP 허용
            allow 10.0.255.8;
            # 외부 공인 IP 허용
            allow 218.235.89.82;
            # 그 외 모든 IP 차단
            deny all;

            # 차단 시 보여줄 에러 페이지 설정
            error_page 403 /403.html;

            # 실제 관리자 서비스(K8s 인그레스)로 연결
```

```
            proxy_pass http://172.16.0.110/admin/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        # [추가] 403 에러 발생 시 K8s 웹 서버에서 파일을 가져오는 통로
        location = /403.html {
            proxy_pass http://172.16.0.110/403.html;
            proxy_set_header Host $host;
            internal; # 외부에서 직접 /403.html로 접속하는 것은 막음
        }
}

# HTTPS(443) 요청을 받으면 HTTP(80)로 던져주는 설정
server {
    listen 443;
    server_name sy-locker-view.com grafana.sy-locker-view.com;

    # 301 리다이렉트를 통해 https -> http로 강제 전환
    return 301 http://$host$request_uri;
}
```

3. K8s-M에서 nginx-conf.yaml파일의 defalut.conf 내용을 수정합니다.

```
# [진짜 IP 추출 설정]
# 모든 사설 대역(Calico, Node, MetalLB)을
# 신뢰 대역으로 추가

set_real_ip_from 10.0.0.0/8;
set_real_ip_from 172.16.0.0/12;
set_real_ip_from 192.168.0.0/16;
real_ip_header X-Forwarded-For;
real_ip_recursive on;
```

4.3.2. 방화벽 설정

시스템 보안을 위해 외부 노출 포트를 최소화하고, 모든 외부 트래픽은 **배스천 호스트(Bastion Host)의 Nginx**를 거쳐 내부망으로 전달되도록 설계하였습니다.

일반 사용자가 접근하지 않는 모든 관리 작업은 내부 전용 **ClusterIP**와 **Bastion Host**를 통해서만 수행되도록 설계하였습니다.

▼ **배스천 호스트의 default.conf**

```
server {
        listen 80 default_server;
        listen [::]:80 default_server;

        server_name sy-locker-view.com www.sy-locker-view.com localhost grafan
```

```
a.sy-locker-view.com;

        root /var/www/html;


        index index.html index.htm index.nginx-debian.html;

        location / {
            # 쿠버네티스 MetalLB(인그레스) IP로 전달
            proxy_pass http://172.16.0.110/;

            # [핵심] 원본 IP를 전달하는 헤더 3인방
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

            # 프록시 관련 추가 설정 (안정성)
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection 'upgrade';
            proxy_cache_bypass $http_upgrade;


            # 타임아웃 방지
            proxy_connect_timeout 90;
            proxy_send_timeout 90;
            proxy_read_timeout 90;

        }

        # 3. 관리자 전용 경로 (IP 제한 및 커스텀 403 적용)
        location /admin/ {
            # [핵심] 오늘 확인한 SY님 PC의 실제 IP 허용
            allow 10.0.255.8;
            # 외부 공인 IP 허용
            allow 218.235.89.82;
            # 그 외 모든 IP 차단
            deny all;

            # 차단 시 보여줄 에러 페이지 설정
            error_page 403 /403.html;

            # 실제 관리자 서비스(K8s 인그레스)로 연결
            proxy_pass http://172.16.0.110/admin/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        # [추가] 403 에러 발생 시 K8s 웹 서버에서 파일을 가져오는 통로
        location = /403.html {
            proxy_pass http://172.16.0.110/403.html;
            proxy_set_header Host $host;
```

```
            internal; # 외부에서 직접 /403.html로 접속하는 것은 막음
        }
}

# HTTPS(443) 요청을 받으면 HTTP(80)로 던져주는 설정
server {
    listen 443;
    server_name sy-locker-view.com grafana.sy-locker-view.com;

    # 301 리다이렉트를 통해 https -> http로 강제 전환
    return 301 http://$host$request_uri;
}
```

▼ 방화벽 및 보안 제어

단계	주체	동작	비고
1. 요청	일반 사용자	`sy-locker-view.com` 접속	공용망을 통한 접근
2. 관문	**배스천 Nginx**	접근 권한 확인 및 `proxy_pass` 실행	**작성된 설정 (ACL/Proxy) 적용 지점**
3. 도달	**MetalLB (L4)**	VIP(`172.16.0.110`)로 트래픽 수신	인프라 접점 및 부하 분산 시작
4. 분류	**Ingress (L7)**	경로 분석 후 해당 서비스로 배송	클러스터 내부 정밀 라우팅

1. **외부 진입점 및 리버스 프록시 (Single Entry Point)**

   일반 사용자는 보안 구역인 내부망이나 MetalLB IP에 직접 접근하지 않고, 배스천 호스트의 Nginx 게이트웨이를 통해 서비스에 안전하게 접속합니다. 이 Nginx 게이트웨이는 인그레스를 통해 배당된 도메인을 통해 연결됩니다.

   - **배스천 Nginx의 중계 역할:** 사용자가 도메인(`sy-locker-view.com`)으로 접속하면, 배스천의 Nginx가 이 요청을 받아 내부 MetalLB VIP(172.16.0.110)로 전달(proxy_pass)합니다.
   - **포트 개방 최적화:** 외부망(Public)에서는 웹 서비스용 80(HTTP), 443(HTTPS) 및 관리용 22(SSH) 포트만 개방했습니다.
   - **트래픽 유도 및 효율화:** `443` 포트로 들어오는 보안 요청을 301 리다이렉트(URL의 영구적 이동을 알리는 상태 코드)를 통해 `80` 포트로 유도하여 클러스터 내부 통신 규격을 단일화했습니다.

2. **경로 기반 보안 제어 및 사용자 식별**

   Nginx 게이트웨이는 요청을 전달하기 전, 경로에 따라 강력한 접근 제어(ACL)를 수행하며 사용자 정보를 보존합니다.

   - **ACL:** `/admin/` 경로는 `allow` 지시문을 통해 인가된 관리자 IP(10.0.255.8 등)만 허용하며, 그 외의 접근은 `deny all` 규칙과 커스텀 403 Forbidden 페이지로 즉시 차단합니다.
   - **헤더 포워딩 (Header Forwarding):** default.conf에서 `proxy_set_header` 설정을 통해 사용자의 실제 IP(X-Real-IP)를 쿠버네티스 내부까지 전달하여 프록시 환경에서도 정확한 로그 분석과 보안 모니터링이 가능하게 합니다.

▼ 4.4. 데이터 흐름 설계

▼ 데이터 흐름도 (Data Flow)

```
[1. 서비스 및 트래픽 흐름 (Service Path)]
사용자 (sy-locker-view.com)
   │ 외부 Nginx (Reverse Proxy/IP ACL)
   ▼ [L4] MetalLB (VIP: 172.16.0.110) ──▶ [L7] Ingress
   │
   └─▶ [WEB] Nginx Pod ──▶ [WAS] FastAPI Pods (V/R/A)
                                  │
                                  ▼ [SVC/Endpoints]
                          MariaDB Master (172.16.0.8: Write)


[2. 로그 수집 및 정적 자원 흐름 (Storage Path)]
Nginx/FastAPI Pods (Logs)          Nginx Pods (Static Assets)
   │                                       ▲
   ▼ [PVC] nfs-log-pvc (5Gi)               │ [PVC] nfs-pvc (10Gi)
   │                                       │
   └──────────▶ [Network] NFS Server (172.16.0.3) ◀──────┘
                                  │
             [/mnt/DATA1/nginx-logs] [/mnt/DATA2/k8s-web]


[3. DB 복제 및 격리 백업 흐름 (Replication & Backup Path)]
MariaDB Master (172.16.0.8)
   │
   ▼ [Sync] Binary Log 전송 (실시간 동기화)
   │
MariaDB Slave (172.16.0.9) ──▶ [Relay Log 재현]
   │
   ▼ [Process] Crontab 실행 (db_backup.sh / mysqldump)
   │
   ▼ [Path] Worker Node 로컬 경로 (/mnt/db_backup)
   │
   ▼ [HostPath] backup-monitor Pod 마운트 (NFS 연동)
   │
   └──────────────▶ NFS Server [/mnt/DATA2/slave/data]
                    (db_full_YYYYMMDD_HHMMSS.sql 보관)
```

4.4.1. 요청 처리 흐름

▼ 1. 서비스 및 트래픽 경로 (Service & Traffic Path)

사용자 ──▶ 외부 Nginx (Reverse Proxy/IP ACL) ──▶ MetalLB (VIP: 172.16.0.110) ──▶ Ingress (L7) ──▶ Nginx Pod (Web) ──▶ FastAPI Pod (WAS) ──▶ MariaDB Master

- **진입 및 보안:** 사용자가 도메인(`sy-locker-view.com`) 접속 시, 외부 Nginx(Reverse Proxy)가 IP ACL 및 301 리다이렉트를 수행한 후 내부로 전달합니다.
- **로드밸런싱 및 라우팅:** 트래픽은 MetalLB(L4 VIP: 172.16.0.110)를 거쳐 Ingress(L7)에서 도메인 및 경로에 따라 적절한 서비스로 분기됩니다.
- **Web-WAS:** 요청 성격에 따라 정적 웹 자원은 Nginx Pod가, API 로직 및 DB 통신은 FastAPI Pod가 전담하여 처리합니다.

- **데이터 접근:** FastAPI는 쿠버네티스 Service/Endpoints 설정을 통해 외부 물리 서버인 MariaDB Master(172.16.0.8)에 접속하여 데이터를 조회 및 변경합니다.

▼ 2. 로그 수집 및 정적 자원 경로 (Storage Path)

Nginx/FastAPI (Logs) ──▶ NFS-Log-PVC ──▶ NFS Server (/DATA1)NFS Server (/DATA2) ──▶ NFS-PVC ──▶ Nginx Pod (Static Assets 서빙)

- **정적 자원 서빙:** Nginx의 웹 정적 파일은 nfs-pvc(10Gi)를 통해 `/mnt/DATA2/k8s-web` 경로에서 Nginx 파드로 마운트되어 사용자에게 서빙됩니다.
- **중앙 집중형 로깅:** Nginx 및 FastAPI 파드에서 발생하는 실시간 로그는 nfs-log-pvc(5Gi)를 통해 NFS 서버의 `/mnt/DATA1/nginx-logs` 경로에 통합 기록됩니다.
- **권한 최적화:** 모든 NFS 접속 요청을 **shared_user(1001)** 권한으로 강제 일치시켜, 다중 노드 환경에서도 권한 충돌(`Permission Denied`) 없이 안정적인 파일 공유가 가능합니다.

▼ 3. DB 복제 및 격리 백업 경로 (Replication & Backup Path)

Master ──▶ Slave (Replication) ──▶ Backup Script (Dump) ──▶ Worker Node (HostPath) ──▶ NFS Server (영구 보관)

- **실시간 복제:** Master DB의 모든 변경 사항은 Binary Log를 통해 MariaDB Slave(172.16.0.9)로 즉시 전송되어 Master의 로그를 Relay Log로 재현하여 실시간 데이터 동기화를 유지합니다.
- **자동 백업 프로세스:** Slave 서버 내부에서 Crontab 스케줄러가 `db_backup.sh`를 주기적으로 실행하여 최신 상태의 `mysqldump` 파일을 생성합니다.
- **격리 및 보관:** 생성된 덤프 파일은 워커 노드의 로컬 경로를 거쳐 backup-monitor Pod의 NFS 마운트를 통해 최종적으로 NFS Server(/mnt/DATA2/slave/data)에 `.sql` 형태로 격리 보관됩니다.

4.4.2. 장애 발생 시 흐름

장애 유형	대응 메커니즘	상세 동작 및 복구 방안
인프라(Bastion) 장애	**Keepalived HA**	Master 장애 시 Backup 서버가 VIP(`172.16.0.100/200`)를 자동 승계하여 외부 접속 유지
데이터베이스 장애	**Master-Slave 승격**	Master 불능 시 Slave를 Master로 승격하거나, **NFS에 저장된 최신 백업본**으로 복구 수행
애플리케이션 장애	**Self-healing**	파드 다운 시 K8s Deployment가 즉시 파드를 재생성하고, Endpoints가 살아있는 IP로 트래픽 즉시 전환
데이터베이스 파손	**백업 데이터 활용**	데이터베이스 파손 시, NFS(/mnt/DATA2/slave/data)에 보관된 `.sql` 덤프 파일을 통해 신규 클러스터에 데이터 복원
스토리지 용량 부족	**LVM 확장**	서비스 중단 없이 `cfdisk`, `pvresize`, `lvextend`를 통해 논리 볼륨 크기를 유연하게 확장

▼ keepalived.conf 설정 (VIP 설정)

서비스 그룹	virtual_router_id	비고
K8s Master HA	**51, 52**	k8s 마스터/슬레이브 공통
Bastion HA	**61**	배스천 마스터/슬레이브 공통

```
sy@bastion:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:8f:b3:e4 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 10.0.105.1/16 brd 10.0.255.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8f:b3e4/64 scope link
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:8f:b3:ee brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 172.16.0.2/24 brd 172.16.0.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet 172.16.0.100/32 scope global ens37
       valid_lft forever preferred_lft forever
    inet 172.16.0.200/32 scope global ens37
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8f:b3ee/64 scope link
       valid_lft forever preferred_lft forever
```

bastion은 VIP가 할당되었다.

```
sy@bastion-s:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:39:09:6f brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 10.0.105.2/16 brd 10.0.255.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe39:96f/64 scope link
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:39:09:79 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 172.16.0.11/24 brd 172.16.0.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe39:979/64 scope link
       valid_lft forever preferred_lft forever
```

bastion-s는 VIP가 할당되지 않았다.

💡 **이중화 & 고가용성(HA)**

- **배스천(Master)**
  - **Bridged (`ens33`):** `10.0.105.1` (외부 접속용 실 IP)
  - **Host-only (`ens37`):** `172.16.0.2` (내부 관리용 실 IP)
    - **VIP (`ens37`에 할당됨):**
      - `172.16.0.100`: 서비스용 VIP
      - `172.16.0.200`: 관리용 VIP

  ⇒ 현재 `bastion`이 **MASTER** 역할을 수행합니다. 할당한 VIP 두 개를 모두 정상적으로 가지고 있습니다.

- **배스천-s(Backup)**
  - **Bridged (`ens33`):** `10.0.105.2` (외부 접속용 실 IP)
  - **Host-only (`ens37`):** `172.16.0.11` (내부 관리용 실 IP)

  ⇒ 현재 VIP가 보이지 않습니다. 이는 **BACKUP** 모드로 대기 중입니다. (Master가 죽으면 이 서버의 `ens37`에 VIP가 나타나게 됩니다.)

▼ 하드디스크 용량 늘리기 (disk_extend.sh)

1. **파티션 크기 확장 (cfdisk 활용)**

```
sudo cfdisk /dev/sda
/dev/sda3
Resize
Write:yes
Quit
```

2. **변경사항 커널에 알리기**

```
sudo partprobe /dev/sda
```

3. **물리 볼륨 다시 인식**

```
sudo pvresize /dev/sda3
```

4. **논리 볼륨 확장**

```
sudo lvextend -l +100%FREE /dev/mapper/ubuntu--vg-ubuntu--lv
```

5. **파일 시스템 사이즈 최종 적용**

```
sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
```

---

5. 기타: 프로젝트 산출물

▼ 5.1. 구현 서비스별 구현 파일 목록(목록번호 부여)

5.1.2. 서비스 구현 순서(명령 포함)

1. **bastion: 인프라 관리 및 고가용성 설정**

   1-1. default.conf (Nginx 프록시 설정)
   1-2. keepalived.conf (VIP 및 HA 이중화 설정)

==========

2. **local: 접속 편의성 및 로컬 환경 설정**

   2-1. config (SSH 점프 프록시 설정)
   2-2. hosts (로컬 호스트 파일 설정)

==========

3. **db-m: 메인 데이터베이스 구성**

   3-1. init.sql (초기 데이터베이스 구조 및 데이터 설정)
   3-2. 마스터-슬레이브 복제.sql

==========

4. **db-s: 백업 및 복제 자동화 설정**

   4-1. db_backup.sh (백업 실행 스크립트)
   4-2. setup_crontab.sh (크론탭 설정 - 정기 백업 스케줄링)
   4-3. db-backup-hostpath.yaml (백업용 볼륨 매니페스트)

==========

5. **k8s-m: 쿠버네티스 프로젝트 디렉토리 (Master)**

5-0.1. keepalived.conf (VIP 및 HA 이중화 설정)
5-0.2. ingress-patch.sh (Ingress CofigMap 설정)

- **5-1. db-configs: DB 백업 자동화**

  5-1.1. db_backup.sh (DB 백업 실행 스크립트-k8s에 임시 저장)

- **5-2. k8s-manifests: 서비스 인프라 매니페스트**

  < 네트워크 및 인프라 구축 >
  5-2.1. metallb.yaml (L4 로드밸런서 VIP 설정)
  5-2.2. nginx-conf.yaml (Nginx 설정 ConfigMap)

  < 스토리지 설정 >
  5-2.3. pv-pvc.yaml (nfs 스토리지 연결)
  5-2.4. nfs-log-pv-pvc.yaml (로그 저장용 볼륨)

  <데이터베이스 연결>
  5-2.5. mariadb-external.yaml (외부 DB 연동 설정)

  < 애플리케이션 배포 >
  5-2.6. deployment.yaml (Pod 배포 및 복제본 관리)
  5-2.7. fastapi.yaml (애플리케이션 상세 설정)
  5-2.8. fastapi-services.yaml (내부 서비스 노출)

  < 외부 접속 노출 >
  5-2.9. ingress.yaml (도메인 기반 라우팅)
  5-2.10. grafana-ingress.yaml (모니터링 대시보드 노출)

- **5-3. nginx: 정적 웹 리소스**

  5-3.1. index.html (메인 화면)
  5-3.2. 403.html (권한 에러 화면)

- **5-4. service-locker: 조회 서비스 (FastAPI)**

  5-4.1. main.py
  5-4.2. Dockerfile
  5-4.3. requirements.txt

- **5-5. service-locker: 조회 서비스 (FastAPI)**

  5-5.1. main.py
  5-5.2. Dockerfile
  5-5.3. requirements.txt

- **5-6. service-admin: 관리자 서비스 (FastAPI)**

  5-6.1. main.py
  5-6.2. Dockerfile
  5-6.3. requirements.txt

==========

6. **장애 대응**

   6-1. disk_extend.sh (하드디스크 용량 늘리기)

▼ 5.2. 서비스별 구현 파일 첨부(목록번호 순서)

프로젝트 압축 파일을 참고해주세요.

GitHub - ParkSoyoung77/locker-project

Contribute to ParkSoyoung77/locker-project development by creating an account on GitHub.

https://github.com/ParkSoyoung77/locker-project

ParkSoyoung77/**locker-project**

1 Contributor 0 Issues 0 Stars 0 Forks